

Министерство образования и науки Российской Федерации
федеральное государственное бюджетное образовательное
учреждение высшего образования
«Алтайский государственный технический университет
им. И.И. Ползунова»

Факультет информационных технологий
Кафедра прикладной математики
Специальность (направление, профиль) ПИ

Курсовой проект
защищен с оценкой _____
_____ Е.В. Егорова
“ ____ ” _____ 2026 г.

КУРСОВОЙ ПРОЕКТ

Разработка программного обеспечения для организации
списка данных об авиарейсах

Пояснительная записка

по дисциплине Программирование
КП 09.03.04.01.000 ПЗ

Студент группы ПИ54 Дворников М.С. 7.05.2024

Руководитель проекта
доцент, к.т.н. Е.В. Егорова

БАРНАУЛ 2026

Министерство науки и высшего образования Российской Федерации
ФГБОУ ВО «Алтайский государственный технический университет
имени И.И. Ползунова»

Факультет информационных технологий

Кафедра «Прикладная математика»

ЗАДАНИЕ

на курсовой проект по дисциплине «Программирование»
студенту группы ПИ-54 Дворникову Марку Сергеевичу

Тема курсового проекта: «Разработка программного обеспечения для
организации списка данных об авиарейсах».

Календарный план работы:

№ этапа	Содержание этапа	Недели семестра
1	Получение задания	1
2	Постановка задачи. Работа с документацией	2
3	Проектирование программы	3-4
4	Реализация программы	5-13
5	Оформление пояснительной записки	14
6	Защита курсового проекта	15-16

Руководитель проекта _____ Егорова Е.В., доцент

Дата выдачи задания «02» февраля 2026 г.

Задание принял к исполнению _____ Дворников М.С.

Содержание

Введение.....	1
1 Обзор предметной области и постановка задачи	2
1.1 Обзор предметной области	2
1.2 Постановка задачи	3
1.2.1 Общее описание задачи.....	3
1.2.2 Требования к функциональности	3
2 Проектирование	5
2.1 Укрупненный алгоритм решения.....	5
2.2 Структура данных.....	5
2.2.1 Структура записи	5
2.2.2 Структура фильтра.....	6
3 Реализация	7
3.1 Выбор средств реализации.....	7
3.2 Структура программы	8
3.3 Состав программы	10
3.3.1 Функции ввода. input_funcs	10
3.3.2 Функции работы с файлами. file_funcs.....	11
3.3.3 Функции для работы со структурами. structure.cpp	11
3.3.4 Общие функции. functions.....	14
Заключение	16
Список использованных источников	18
Приложение А	19
П.А.1 Код input_funcs.h	19
П.А.2 Код input_funcs.cpp	19

П.А.3 Код file_funcs.h	24
П.А.4 Код file_funcs.cpp	24
П.А.5 Код structure.h	28
П.А.6 Код structure.cpp	30
П.А.7 Код functions.h	44
П.А.8 Код functions.cpp.....	44
П.А.9 Код Mark_flights.cpp	48
Приложение Б. Тесты программы.....	51

Введение

Для проектов самой разной направленности всегда была актуальна проблема хранения информации, ее структуризации и возможности эффективного использования. Сегодня эта задача может быть значительно упрощена за счет работы с электронными таблицами и базами данных.

Работа с пользовательскими данными – это распространенный вид деятельности в сфере информационных технологий, предполагающий большой объем работы и большую ответственность. Поэтому каждому разработчику прикладного программного обеспечения необходимо получить опыт в реализации подобных проектов.

Цель данной работы: изучение принципов разработки консольных приложений для организации работы с большими наборами структурированных данных.

Главная задача: написать программу, работающую в консольном режиме и позволяющую организовать сбор, хранение и обработку структурированных данных определенной направленности.

1 Обзор предметной области и постановка задачи

1.1 Обзор предметной области

В современном мире информация стала одним из самых ценных ресурсов. Особенно это касается структурированных данных – информации, организованной по определённым правилам и предназначенной для хранения, поиска и обработки. Для работы с такими данными используются информационные системы, одной из разновидностей которых являются базы данных.

База данных (БД) — это совокупность данных, организованная по определённым правилам, обеспечивающая хранение, доступ и представление информации в удобном для пользователя виде. Простейшим примером базы данных является структурированный список, где каждая строка (запись) содержит информацию об одном объекте, а столбцы (поля) – характеристики этих объектов. Такие списки широко применяются на практике: расписание движения поездов и самолётов, каталог товаров, список студентов, база контактов и т.д.

Программное обеспечение для работы со структурированными списками должно решать несколько типовых задач:

- ввод новых данных;
- вывод данных в удобочитаемом виде;
- корректировка (редактирование) существующих данных;
- удаление устаревших или ошибочных данных;
- поиск и фильтрация данных по определённым критериям.

Особую важность приобретает вопрос долговременного хранения данных. Программа не может полагаться только на оперативную память, так как при выключении компьютера все данные теряются. Поэтому данные должны сохраняться в файле на диске. При этом работа с файлом должна

быть организована эффективно: перенос всего файла в оперативную память для выполнения каждой операции (как это часто делается в учебных примерах) становится недопустимым при больших объёмах данных. Вместо этого необходимо выполнять операции непосредственно в файле или с использованием вспомогательных файлов.

Таким образом, разработка программы, позволяющей организовать хранение, редактирование и поиск данных в структурированном списке с сохранением информации в файле, является актуальной задачей, имеющей как учебную, так и практическую ценность.

1.2 Постановка задачи

1.2.1 Общее описание задачи

Требуется разработать программное обеспечение, которое обеспечивает организацию, хранение и обработку структурированного списка данных об авиарейсах. Каждая запись содержит:

- номер рейса;
- тип самолета;
- пункт назначения;
- дни отправления;
- время вылета;
- время прилета;
- цена билета.

Например: 121 Ту-154 Москва 1,3,5 10.00 14.00 5000.20

1.2.2 Требования к функциональности

Программа должна обеспечивать выполнение следующих функций:

- Ввод и корректировка таблицы:
 - Ввод таблицы;
 - Изменение записей;
 - Удаление записей;

- запрос по любой комбинации любых полей;
- Вывод таблицы на экран.

В программе следует организовать необходимые защиты от неправильного ввода данных так, чтобы ни при каких обстоятельствах программа не выдала ошибку. В программе должен быть реализован дружественный интерфейс, предполагающий диалоговое меню для пользователя и удобный, однозначно понятный ввод данных и вывод данных.

2 Проектирование

2.1 Укрупненный алгоритм решения

При реализации программного обеспечения используется метод нисходящего проектирования и модульное программирование. Программа работает в диалоговом режиме.

При входе в программу пользователь сначала попадает на первичную настройку БД.

Далее открывается меню, которое позволяет выбрать дальнейшие действия:

- Ввод и корректировка таблицы;
- Установка фильтров (запрос по любой комбинации любых полей);
- Изменение пути до файла;
- Вывод таблицы на экран.

Выбор «Ввод и корректировка таблицы» будет перенаправлять пользователя в другое меню:

- Ввод таблицы;
- Изменение записей;
- Удаление записей;

2.2 Структура данных

2.2.1 Структура записи

Запись об одном авиарейсе в программе хранится в структуре Flight. Каждая структура имеет поля:

1. `fnum` – переменная целого типа для хранения номера рейса;
2. `name` – символьный массив (строка) для хранения типа самолёта;
3. `dest` – символьный массив (строка) для хранения пункта назначения;
4. `days` – целочисленный массив для хранения чисел от 1 до 7, которые обозначают дни недели (1 – понедельник, 2 – вторник и т.д.);
5. `dep_time` – переменная целого типа для хранения времени вылета в минутах;

6. `arr_time` – переменная целого типа для хранения времени прилёта в минутах;
7. `price` – переменная типа числа двойной точности для хранения цены билета

Данные хранятся в файле `table.csv` в формате CSV (Comma-Separated Values) с разделителем «точка с запятой» (;). Каждая запись соответствует одной строке файла.

Например, запись «121 Ту-154 Москва 1,3,5 10.00 14.00 5000.20» будет иметь следующие данные:

- `fnum = 121;`
- `name = "Ту-154";`
- `dest = "Москва";`
- `days = {1, 3, 5, 0, 0, 0, 0};`
- `dep_time = 600;`
- `arr_time = 840;`
- `price = 5000.20;`

При этом `table.csv` будет хранить строчку:

«121;Ту-154;Москва;135000;600;800;5000.20»

2.2.2 Структура фильтра

Фильтры хранятся в массиве и представляют собой структуру `Flight_filter`. Она имеет те же поля, что и структура `Flight`, но к ним добавляются ещё два:

8. `apply` – целочисленный массив для хранения 0/1 в зависимости от того, на какое поле нужно применять фильтр, а на какое нет. Для некоторых полей (`dep_time`, `arr_time`, `price`) значения массива могут быть числами от 0 до 5 – в таком случае они означают операцию сравнения;
9. `logic` – целочисленный массив для хранения 0/1 в зависимости от того, какой логический оператор нужно применить при сравнении записи с фильтром.

3 Реализация

3.1 Выбор средств реализации

Для разработки программного обеспечения был выбран язык программирования С. Язык С предоставляет богатые возможности для работы с файлами, позволяет эффективно управлять памятью и реализовывать структуры данных, что необходимо для выполнения поставленной задачи.

Разработка велась в интегрированной среде разработки Microsoft Visual Studio 2022 с использованием компилятора MSVC (Microsoft Visual C++). Приложение является консольным (не использует графический интерфейс пользователя), что соответствует требованиям курсового проекта и обеспечивает простоту разработки и отладки.

Для полноценного функционирования программы требуется:

- ОС Windows XP или выше;
- Оперативная память 10МБ или выше;
- Жесткий диск: 10МБ или выше для хранения программы и дополнительное место для хранения вводимых в базу данных;
- Наличие стандартных устройств ввода-вывода – клавиатура и монитор (дисплей)

3.2 Структура программы

Основная структура программы представлена на рисунке 1:

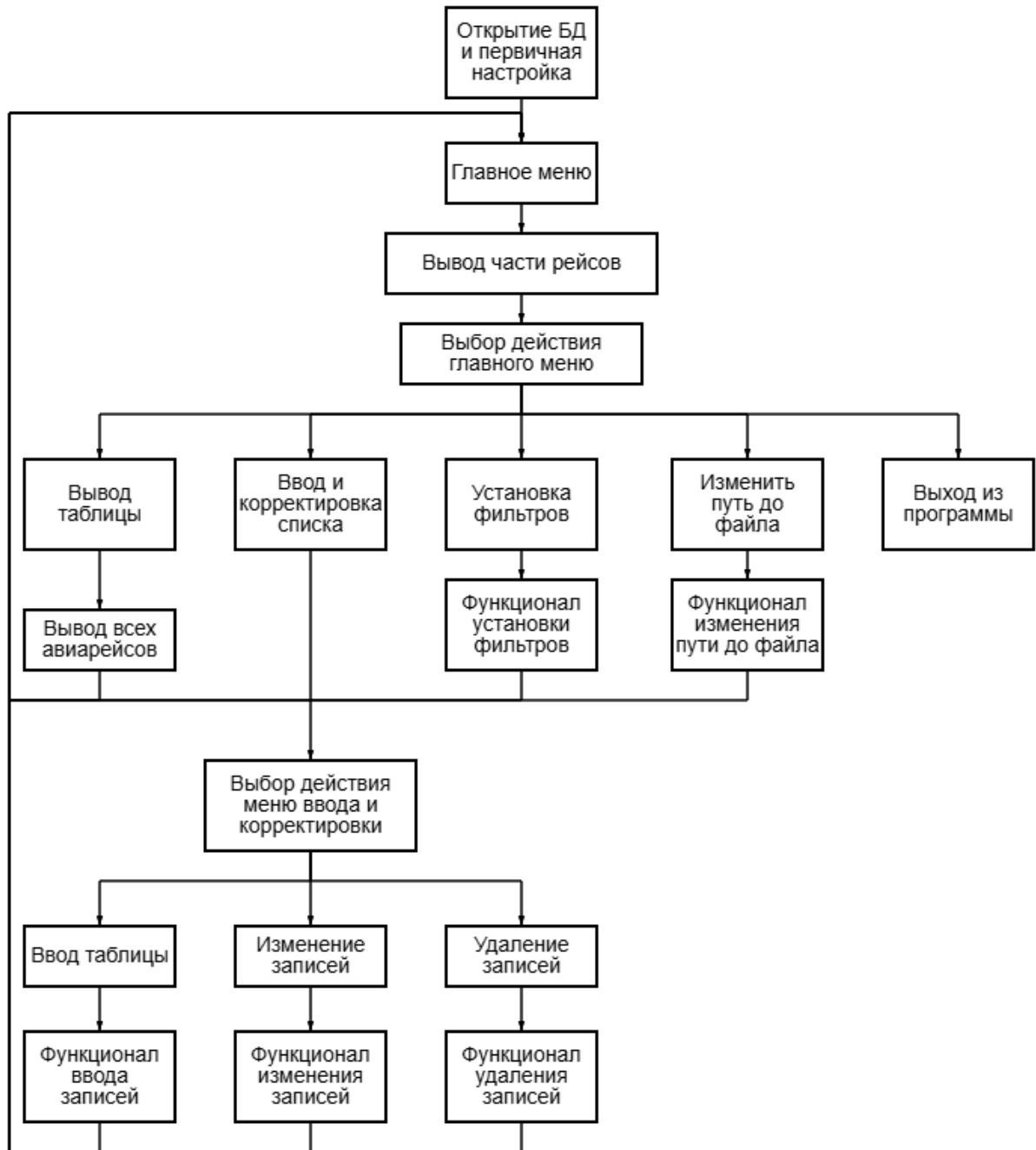


Рисунок 1 – Основная структура программы

- Открытие БД и первичная настройка – проверка на существования нужных для работы программы файлов;
- Вывод всех авиарейсов – чтение БД и вывод авиарейсов, которые подходят под заданные фильтры;

- Функционал установки фильтров – программа выводит все авиарейсы, которые подходят под заданные фильтры, все фильтры (если таких нет, то «не используется») и в диалоговом режиме получает информацию об установке (добавлении/изменении/удалении) фильтров;
- Функционал изменения пути до файла – программа в диалоговом режиме получает информацию об изменении пути до файла;
- Функционал ввода записей – программа в диалоговом режиме заполняет структуру по полям, а затем сохраняет её в БД;
- Функционал изменения записей – программа в диалоговом режиме получает номера записей, которые нужно изменить, а затем меняет их по введённому шаблону;
- Функционал удаления записей – программа в диалоговом режиме получает номера записей, которые нужно удалить, и выполняет удаление.

Работа с файлом данных организована методом последовательного доступа. При удалении записи создаётся вспомогательный файл, в который копируются все записи исходного файла, кроме удаляемой, после чего исходный файл заменяется. При редактировании записи используется аналогичный подход: записи построчно копируются во вспомогательный файл с одновременной заменой редактируемой записи на новую. Такой способ исключает необходимость загрузки всего файла в массив структур и позволяет работать с файлами любого размера.

Для создания запроса по любой комбинации любых полей используются фильтры. Основная идея механизма заключается в том, что пользователь независимо настраивает фильтр для каждого поля, при этом для одного поля можно задать несколько условий с разными логическими операторами (И / ИЛИ). Все активные фильтры хранятся в массиве.

Путь до файла БД всегда хранится в файле `flights_data/settings.txt`, что позволяет менять путь до файла БД и не терять данные.

3.3 Состав программы

Для удобного написания алгоритмов было принято решение разбить программу на модулей по назначению функций. Каждый модуль содержит два файла – .h (для прототипов) и .cpp для определений функций.

В главном файле Mark_flights.cpp содержится лишь одна функция – main (прототип: int main();), с которой начинается выполнение всей программы.

3.3.1 Функции ввода. input_funcs

Назначения:

- input – функция для сохранения введённого текста в переменную и первичная обработка (например, очистка потока, если требуется);
- int_input – функция для «безопасного» ввода переменной типа int;
- double_input – функция для «безопасного» ввода переменной типа double;
- number_array_input – функция для ввода массива цифр;
- int_array_input – функция для ввода массива натуральных чисел без повторений;
- input_time – функция для ввода времени в формате xx.xx или xx:xx и преобразование в int.

Прототипы:

```
// Сохранение ввода в переменную и первичная обработка
int input(char buffer[], int mem, int *overflow = NULL);

// Безопасный ввод int
int int_input(int *buffer);

// Безопасный ввод double
int double_input(double *buffer);

// Безопасный ввод массива цифр
int number_array_input(int array[], int *len, int start = 1, int end = 7);

// Безопасный ввод массива натуральных чисел без повторений
int int_array_input(int **p_array, int len);

// Ввод времени в формате xx.xx или xx:xx и преобразование в int
int input_time(int *buffer);
```

3.3.2 Функции работы с файлами. file_funcs

Назначения:

- correct_name – функция для проверки, есть ли название файла (table.csv) в указанном пути (вынесена для удобства);
- get_file – функция для получения пути до файла БД из файла flights_data/settings.txt (а так же многочисленные проверки этого пути);
- change_path – функция для взаимодействия с пользователем с целью изменения пути до файла;
- read_line – функция для чтения файла в структуру;
- write_line – функция для записи структуры в файл;
- count_lines – функция для подсчёта строк в файле table.csv;

Прототипы:

```
// Проверка, влезло ли название файла FILE_NAME в строку
int correct_name(char *file_path);

// Получить путь до файла, в который нужно напisyвать данные
int get_file(char *file_path, int buffer_len = FILE_NAME_LEN);

// Изменить путь до файла, в который нужно напisyвать данные
int change_path();

// Записать одну строчку файла в структуру
int read_line(FILE *table, Flight *flight);

// Записать одну структуру таблицы в файл
int write_line(FILE *table, Flight flight);

// Посчитать количество подходящих под фильтр строк
int count_lines(FILE *table, Flight_filter filters[], int *len, int *fil_len);
```

3.3.3 Функции для работы со структурами. structure.cpp

В этом файле есть объявление структур:

```
// Структура информации о полёте
typedef struct {
    int fnum;           // Номер рейса           (flight number)
    char name[TEXT_LEN]; // Тип самолёта       (название модели)
    char dest[TEXT_LEN]; // Пункт назначения   (destination)
    int days[7];        // Дни отправления
    int dep_time;       // Время вылета в минутах (departure time)
    int arr_time;       // Время прилёта в минутах (arrival time)
    double price;       // Цена билета         (десятичная дробь)
} Flight;
```

```
// Структура для фильтров записей
typedef struct {
    int fnum;           // Номер рейса           (flight number)
    char name[TEXT_LEN]; // Тип самолёта       (название модели)
    char dest[TEXT_LEN]; // Пункт назначения   (destination)
    int days[7];        // Дни отправления
    int dep_time;        // Время вылета в минутах (departure time)
    int arr_time;        // Время прилёта в минутах (arrival time)
    double price;        // Цена билета         (десятичная дробь)

    int apply[FIELDS_NUM]; // Какие столбцы нужно фильтровать (и каким образом)
    int logic[FIELDS_NUM]; // Какую операцию нужно применять при сложении
} Flight_filter;
// фильтров
```

Назначения функций:

- `input_text_field` – функция для ввода текстовых полей (вынесена для удобства);
- `input_time_field` – функция для ввода поля, которое содержит время (вынесена для удобства);
- `input_fnum`, `input_name`, `input_dest`, `input_days`, `input_dep_time`, `input_arr_time`, `input_price` – функции для ввода соответствующих полей структуры;
- `input_flight` – функция для ввода полной структуры;
- `print_table_line` – функция для печати на экран строки таблички (например, границы между двумя записями);
- `print_head` – функция для печати на экран шапки таблички;
- `print_blank` – функция для печати на экран «пустой строки» - строки, где в каждой колонке стоит «...»;
- `print_flight` – функция для печати на экран одной структуры;
- `print_flights` – функция для печати на экран всех подходящих под фильтры структур а так же информацию о успешности чтения (например, что таблица пустая);
- `compare_flight` – функция для проверки, подходит ли запись под фильтры;
- `print_filter` – функция для печати на экран заданного поля одного фильтра;

- `print_filters` – функция для печати на экран полной информации о фильтрах;
- `set_filter` – функция для взаимодействия с пользователем с целью установки фильтра;
- `open_dable` – функция для открытия для записи временного файла `dable.csv` и открытия для чтения файла `table.csv`;
- `edit_all_filtered` – функция для взаимодействия с пользователем с целью изменения всех подходящих под фильтры записей;
- `edit_from_array` – функция для взаимодействия с пользователем с целью изменения всех выбранных ранее записей;
- `delete_all_filtered` – функция для удаления всех подходящих под фильтры записей;
- `edit_from_array` – функция для удаления всех выбранных ранее записей;

Прототипы функций:

```
// Ввод структуры
// Ввод поля fnum
int input_fnum(Flight *pointer);

// Ввод текстового поля
int input_text_field(char *field, const char *info);
// Ввод поля name
int input_name(Flight *pointer);
// Ввод поля dest
int input_dest(Flight *pointer);

// Ввод поля days
int input_days(Flight *pointer);

// Ввод временного поля
int input_time_field(int *field, const char *info);
// Ввод поля dep_time
int input_dep_time(Flight *pointer);
// Ввод поля arr_time
int input_arr_time(Flight *pointer);

// Ввод поля price
int input_price(Flight *pointer);

// Ввод полной структуры
int input_flight(Flight *pointer);

// Вывод структур
// Печать части таблицы без данных
```

```

void print_table_line(char vert = '+', char horiz = '-', int cols_num = FIELDS_NUM +
1, int cols_wide = COLS_WIDE);
// Печать шапки таблицы
void print_head();
// Печать "пустой строки"
void print_blank(int cols_num = FIELDS_NUM + 1, int cols_wide = COLS_WIDE);

// Вывод одной записи
void print_flight(Flight flight, int cols_wide = COLS_WIDE);
// Вывод всех записей
int print_flights(Flight_filter filters[], int *len, int *fil_len, int full_info =
0);

// Работа с фильтрами
// Соответствует ли запись всем заданным фильтрам
int compare_flight(Flight flight, Flight_filter filters[]);

// Вывод заданного поля фильтра
int print_filter(Flight_filter filter, int field, int is_first);

// Вывод меню с фильтрами
int print_filters(Flight_filter filters[]);

// Установка фильтра
int set_filter(Flight_filter *p_filter, int field, int is_first = 0);

// Удаление и изменение записей
// Открыть текущий файл table.csv для чтения и dable.csv для записи
int open_dable(char *file_path, char *new_file_path, FILE **new_table, FILE
**old_table);

// Изменение одной структуры
int edit_all_filtered(char *file_path, Flight_filter filters[]);
// Изменение всех записей по фильтру
int edit_from_array(char *file_path, int len, int *to_edit, int edit_len);

// Удалить все подходящие под фильтр записи
int delete_all_filtered(char *file_path, Flight_filter filters[]);
// Удалить все записи, чьи номера есть в массиве
int delete_from_array(char *file_path, int len, int *to_del, int del_len);

```

3.3.4 Общие функции. functions

Назначения:

- `user_prepare` – функция для запуска первичных настроек БД и, если программа запускается впервые, взаимодействие с пользователем с целью установки пути до файла;
- `get_first_action` – функция для вывода вариантов действий в главном меню и ввод номера действия;
- `get_edit_action` – функция для вывода вариантов действий меню «ввод и корректировка списка» и ввод номера действия;

- edit_menu – функция взаимодействия с пользователем с целью изменения записей;
- delete_menu – функция взаимодействия с пользователем с целью удаления записей;
- filters_menu – функция взаимодействия с пользователем с целью установки (добавления/изменения/удаления) фильтров;

Прототипы:

```
// Подготовка пользователя
int user_prepare(char *file_path);

// Выбор первичного действия
int get_first_action(Flight_filter filters[]);

// Выбор действия по вводу и корректировке списка
int get_edit_action(Flight_filter filters[]);

// Меню "Изменение записей"
int edit_menu(char *file_path, Flight_filter filters[]);

// Меню "Удаление записей"
int delete_menu(char *file_path, Flight_filter filters[]);

// Меню "Установка фильтров"
int filters_menu(Flight_filter filters[]);
```

Заключение

В результате выполнения данного курсового проекта разработано программное обеспечение, позволяющее после изучения краткой по объёму документации эффективно работать в диалоговом режиме с базой данных об авиарейсах. Реализован ввод данных с защитой от некорректного ввода, табличный вывод структурированных данных, а также гибкая система фильтрации записей по любому из полей с использованием логических операторов «И» и «ИЛИ».

В процессе выполнения задания были подробно изучены методы работы с файлами на языке С. В соответствии с требованием курсового проекта все операции (добавление, удаление, редактирование) выполняются без загрузки всего файла в оперативную память — используется последовательный доступ через вспомогательный файл, что позволяет обрабатывать базы данных любого размера.

Основные преимущества разработанного приложения:

- Интуитивно понятное иерархическое меню;
- Возможность в любой момент закончить ввод и вернуться в главное меню;
- Быстрая и эффективная обработка запросов с использованием фильтров;
- Возможность оперативного добавления, редактирования и удаления записей;
- Наличие механизма фильтрации с поддержкой сложных логических условий;
- Возможность изменения пути к файлу базы данных без перекомпиляции программы;
- Возможность работы с разными БД путём изменения пути до файла.

Основные недостатки приложения:

- Для работы необходимы специально отформатированные файлы (CSV с разделителем «;»);
- Строгие названия файлов (table.csv);
- Фильтры хранятся в оперативной памяти;
- Отсутствие графического интерфейса (только консольный режим).

Возможные направления дальнейшего усовершенствования:

- Расширение функциональности для работы с несколькими таблицами одновременно;
- Хранение названия файлов вместе с путём до него;
- Хранение фильтров в отдельном файле;
- Сохранение результатов запросов в отдельные файлы;
- Добавление возможности экспорта данных в другие форматы (например, JSON, XML);
- Разработка графического пользовательского интерфейса;
- Реализация функции печати списка (вывод на принтер).

Список использованных источников

- 1 Егорова Е.В. Программирование на языке СИ. Учебное пособие/Алт. Госуд. Технич. Ун-т им. И.И.Ползунова.-Барнаул:2013.-184с.
- 2 “MSDN” – информационный сервис для разработчиков [Электронный ресурс] – Режим доступа: <http://msdn.microsoft.com/ru-ru/ms348103/library>, свободный.
- 3 Онлайн справочник программиста на С и С++ [Электронный ресурс]. Режим доступа: <http://www.c-cpp.ru/>, свободный.

Приложение А

Исходный код программы

П.А.1 Код input_funcs.h

```
#pragma once

#define CANCEL "###" // Отмена ввода

// Сохранение ввода в переменную и первичная обработка
int input(char buffer[], int mem, int *overflow = NULL);

// Безопасный ввод int
int int_input(int *buffer);

// Безопасный ввод double
int double_input(double *buffer);

// Безопасный ввод массива цифр
int number_array_input(int array[], int *len, int start = 1, int end = 7);

// Безопасный ввод массива натуральных чисел без повторений
int int_array_input(int **p_array, int len);

// Ввод времени в формате xx.xx или xx:xx и преобразование в int
int input_time(int *buffer);
```

П.А.2 Код input_funcs.cpp

```
#define _CRT_SECURE_NO_WARNINGS

#include <stdio.h> // Для fgets, getchar, stdin
#include <string.h> // Для strlen, strcmp, strspn, strtok
#include <stdlib.h> // Для strtol, strtod, INT_MIN, INT_MAX, calloc, free
#include <math.h> // Для HUGE_VAL, log10

#include "input_funcs.h"

// Сохранение ввода в переменную и первичная обработка
int input(char buffer[], int mem, int *overflow) {
    /*
    buffer: указатель на строку, в которую нужно записать ввод
    mem: количество памяти, выделенное на строку (максимум mem - 1 символов)
    overflow: указатель на переменную для подсчёта символов, которые не влезли
    Возвращает:
        0: успешное чтение
        1: выход из ввода по сочетанию CANCEL
        2: пустая строка (сразу же нажат "Enter" или только пробелы)
    */
    fgets(buffer, mem, stdin);
    // fgets ставит на место buffer[mem - 1] символ '\0'
    int len = strlen(buffer);

    // Очищаем вводной поток, если строчка не влезла
    int counter = 0; // Для подсчёта лишних символов
    if (buffer[len - 1] != '\n')
        while (getchar() != '\n') counter++;
    else // Удаляем '\n' с конца
        buffer[--len] = '\0';
```

```

    if (overflow != NULL) *overflow = counter;

    // Выход из ввода по сочетанию CANCEL
    if (!strcmp(buffer, CANCEL)) return 1;
    // Пустая строка (сразу же нажат "Enter" иди только пробелы)
    if (len == 0 || strspn(buffer, " ") == strlen(buffer)) return 2;

    return 0;
}

// Безопасный ввод int
int int_input(int *buffer) {
    /*
    buffer: указатель на int, в который нужно записать ввод
    Возвращает:
        0: успешное чтение
        1: выход из ввода по сочетанию CANCEL
        2: пустая строка (сразу же нажат "Enter" иди только пробелы)
        3: есть непрочитанный символ
        4: значение ввода выходит за границы int
    */
    char user_input[33]; // Буфер для ввода пользователя
    int input_res = input(user_input, 33);
    if (input_res) return input_res; // Выход из ввода или пустая строка

    // Переводим в int
    long num; // Число пользователя
    char* endptr; // Указатель на первый непрочитанный символ
    num = strtol(user_input, &endptr, 10);

    // Если есть непрочитанный символ – результат 3
    if (*endptr != '\0') return 3;
    // Если значение выходит за границы – результат 4
    else if (num <= INT_MIN || INT_MAX <= num) return 4;

    *buffer = (int)num;
    return 0;
}

// Безопасный ввод double
int double_input(double *buffer) {
    /*
    buffer: указатель на double, в который нужно записать ввод
    Возвращает:
        0: успешное чтение
        1: выход из ввода по сочетанию CANCEL
        2: пустая строка (сразу же нажат "Enter" иди только пробелы)
        3: есть непрочитанный символ
        4: значение ввода выходит за границы double
    */
    char user_input[33]; // Буфер для ввода пользователя
    int input_res = input(user_input, 33);
    if (input_res) return input_res; // Выход из ввода или пустая строка

    // Переводим в double
    double num; // Число пользователя
    char* endptr; // Указатель на первый непрочитанный символ
    num = strtod(user_input, &endptr);

    // Если есть непрочитанный символ – результат 3
    if (*endptr != '\0') return 3;

```



```

// Если значение выходит за границы - результат 4
else if (num == HUGE_VAL || num == -HUGE_VAL) return 4;

*buffer = num;
return 0;
}

// Безопасный ввод массива цифр
int number_array_input(int array[], int *len, int start, int end) {
/*
array: массив для хранения цифр
len: указатель на переменную, в которую сохраним длину массива
start: минимальная цифра массива
end: максимальная цифра массива

Возвращает:
0: успешное чтение
1: выход из ввода по сочетанию CANCEL
2: пустая строка (сразу же нажат "Enter" или только пробелы)
3: некорректный ввод
*/
char user_input[20]; // Буфер для ввода
int input_res = input(user_input, 20);
if (input_res) return input_res; // Выход из ввода или пустая строка

// Сохраним все допустимые цифры
char symbols[30]; // Массив всех допустимых цифр
int last_ind = 0; // Индекс последнего элемента массива

for (int i = start; i <= end && last_ind < 10; i++)
    symbols[last_ind++] = '0' + i;
symbols[last_ind++] = ' '; symbols[last_ind++] = ','; symbols[last_ind++] =
'\0';

// Проверка на лишние символы - результат 3
if (strspn(user_input, symbols) != strlen(user_input)) return 3;

char* number; // Указатель на цифру для strtok
number = strtok(user_input, " ,");
last_ind = 0; // Обнуляем счётчик

if ((number == NULL) || (strlen(number) > 1)) return 3;
array[last_ind++] = *number - '0';
while ((number = strtok(NULL, " ,")) != NULL) {
    if (strlen(number) > 1) return 3;
    int cur_number = *number - '0';
    // Проверим, есть ли эта цифра в списке
    int in_array = 0;
    for (int i = 0; i < last_ind; i++)
        if (array[i] == cur_number) in_array = 1;
    if (!in_array) array[last_ind++] = cur_number;
}
*len = last_ind; // Сохраняем длину

// Сортируем пузырьком
for (int k = 1; k < last_ind; k++)
    for (int i = 0; i < last_ind - k; i++)
        if (array[i] > array[i + 1]) {
            int buffer = array[i];
            array[i] = array[i + 1];
            array[i + 1] = buffer;
        }
}

```

```

    return 0;
}

// Безопасный ввод массива натуральных чисел без повторений
int int_array_input(int **p_array, int len) {
    /*
    p_array: указатель на массив для чисел (память выделяется внутри функции)
    len: сколько всего может быть чисел (т.е. максимально возможное число)

    После работы функции получается массив (n1, n2, ..., 0).
    0 есть всегда и служит концом массива (т.к. числа только натуральные).

    Если 0 стоит первым, то пользователь ввёл "полный массив"
    "Полный массив" эквивалентен массиву [1, 2, ..., len - 1, len, 0]
    При этом "полный массив" выглядит как [0] для экономии памяти

    Возвращает:
    -1: ошибка выделения памяти
    0: успешное чтение массива
    1: выход из ввода по сочетанию CANCEL
    2: пустая строка (сразу же нажат "Enter" иди только пробелы)
    3: некорректный ввод
    4: строчка не влезла
    */
    int buffer_len = len * ((int)log10(len) + 3) + 4; // Выделяем память с
    расчётом:
    // кол-во чисел * (длина максимального числа + 2 знака на разделение ", ") + 4
    // запасных символа
    char* user_input = (char*)calloc(buffer_len, sizeof(char));
    if (user_input == NULL) return -1;
    int overflow; // Сколько символов не влезло
    int input_res = input(user_input, buffer_len, &overflow);

    // Строчка не влезла
    if (overflow) {
        free(user_input);
        return 4;
    }
    if (input_res) { // Выход из ввода или пустая строка
        free(user_input);
        return input_res;
    }
    if (strspn(user_input, "0123456789") != strlen(user_input)) { // лишние
    символы - результат 3
        free(user_input);
        return 3;
    }
    // Проверяем на "0" - "полный массив"
    if (!strcmp(user_input, "0")) {
        free(user_input);
        if ((*p_array = (int*)calloc(1, sizeof(int))) == NULL) return -1;
        **p_array = 0; // Ставим 0 в начало массива
        return 0;
    }
    else { // В ином случае сразу создаём новый массив
        *p_array = (int*)calloc(len + 1, sizeof(int)); // Массив для номеров
        if (*p_array == NULL) {
            free(user_input);
            return -1;
        }
    }
}
// Начинаем разбирать строчку на номера записей

```

```

int last_ind = 0;    // Индекс последнего элемента
char* number;       // Строка-число для strtok
char* endptr;       // Указатель на символ для strtol

number = strtok(user_input, " ,");
if (number == NULL) {
    free(user_input);
    free(*p_array);
    return 3;
}
do {
    long cur_num_long = strtol(number, &endptr, 10);
    // Проверяем на корректность числа
    if (cur_num_long < 1 || len < cur_num_long) continue;
    int cur_num = (int)cur_num_long;
    // Проверим, есть ли число в массиве
    int in_list = 0;
    for (int i = 0; i < last_ind; i++)
        if ((*p_array)[i] == cur_num) in_list = 1;
    if (!in_list) (*p_array)[last_ind++] = cur_num;
} while ((number = strtok(NULL, " ,")) != NULL);
free(user_input);

// Нет ни одного корректного числа - код 3
if (last_ind == 0) {
    free(*p_array);
    return 3;
}

// Сортируем массив по убыванию пузырьком
for (int k = 1; k < last_ind; k++)
    for (int i = 0; i < last_ind - k; i++)
        if ((*p_array)[i] > (*p_array)[i + 1]) {
            int buffer = (*p_array)[i];
            (*p_array)[i] = (*p_array)[i + 1];
            (*p_array)[i + 1] = buffer;
        }
*(*p_array + last_ind) = 0; // Ставим в конце 0
return 0;
}

// Ввод времени в формате xx.xx или xx:xx и преобразование в int
int input_time(int *buffer) {
    /*
    Возвращает:
    0: успешное чтение массива
    1: выход из ввода по сочетанию CANCEL
    3: некорректный ввод
    */
    int time[2]; // Массив для времени (часы и минуты)
    char user_input[6]; // Буфер для ввода
    int input_res; // Результат ввода

    do input_res = input(user_input, 6);
    while (input_res == 2); // Пока не получим непустую строку
    if (input_res == 1) return 1; // Выход из ввода

    // Проверим на лишние символы
    if (strspn(user_input, " .:0123456789") != strlen(user_input)) return 3;
    char* parts[2]; // Разделим ввод на несколько частей
    parts[0] = strtok(user_input, " .:");
    parts[1] = strtok(NULL, " .:");

```

```

// Строка должна поделиться на две части, а третьей быть NULL
if ((parts[1] == NULL) || (strtok(NULL, " .:") != NULL)) return 3;

// Длина первой части должна быть 1 или 2
int part_len = strlen(parts[0]);
if (part_len == 1)
    time[0] = parts[0][0] - '0';
else if (part_len == 2)
    time[0] = (parts[0][0] - '0') * 10 + parts[0][1] - '0';
else return 3;
// Длина второй всегда 2
if (strlen(parts[1]) == 2)
    time[1] = (parts[1][0] - '0') * 10 + parts[1][1] - '0';
else return 3;

// Проверим на корректные значения
if (time[0] > 23 || time[1] > 59) return 3;

*buffer = time[0] * 60 + time[1];
return 0;
}

```

П.А.3 Код file_funcs.h

```

#pragma once

#define DIR_NAME "flights_data"           // Название папки со всеми данными
#define FILE_NAME "table.csv"             // Название файла с таблицей
#define FILE_NAME_LEN 128                 // Максимальная длина названия файла
#define CONF_NAME "settings.txt"          // Название файла с конфигурацией

#include <stdint.h>           // Для FILE
#include "structure.h"        // Для Flight, Flight_filter

// Проверка, влезло ли название файла FILE_NAME в строку
int correct_name(char *file_path);

// Получить путь до файла, в который нужно напisyвать данные
int get_file(char *file_path, int buffer_len = FILE_NAME_LEN);

// Изменить путь до файла, в который нужно напisyвать данные
int change_path();

// Записать одну строку файла в структуру
int read_line(FILE *table, Flight *flight);

// Записать одну структуру таблицы в файл
int write_line(FILE *table, Flight flight);

// Посчитать количество подходящих под фильтр строк
int count_lines(FILE *table, Flight_filter filters[], int *len, int *fil_len);

```

П.А.4 Код file_funcs.cpp

```

#define _CRT_SECURE_NO_WARNINGS

#include <stdio.h>           // Для FILE, fopen, fclose, fgets, fputs, printf,
remove
#include <string.h>          // Для strlen, strcat, strstr, _strnset
#include <direct.h>          // Для _mkdir

```

```

#include "file_funcs.h"
#include "structure.h" // Для FIELDS_NUM, Flight, Flight_filter
#include "input_funcs.h" // Для input, int_input

// Проверка, влезло ли название файла FILE_NAME в строку
int correct_name(char *file_path) {
    char* file_pntr = strstr(file_path, FILE_NAME); // Указатель на начало
названия файла в строке
    if (file_pntr == NULL)
        return 0; // Если в строке вообще нет названия файла
    if (file_pntr - file_path + strlen(FILE_NAME) != strlen(file_path))
        return 0; // Если название файла не в конце
    return 1;
}

// Получить путь до файла, в который нужно напisyвать данные
int get_file(char *file_path, int buffer_len) {
    /*
    Возвращает:
        -1: У пользователя нет прав даже для папки DIR_NAME
        0: Путь до файла успешно получен
        1: Слишком длинный путь (кто-то вручную поменял файл) - запись
произойдёт по пути DIR_NAME/FILE_NAME
    */
    _mkdir(DIR_NAME); // Пробуем создать папку на случай, если её нету
    FILE* settings = fopen(DIR_NAME "/" CONF_NAME, "a+");
    if (!settings) return -1;
    rewind(settings);

    // Если файл пустой, то возвращаем путь DIR_NAME/FILE_NAME
    if (fgets(file_path, buffer_len, settings) == NULL) {
        strcpy(file_path, DIR_NAME "/" FILE_NAME);
        fclose(settings);
        return 0;
    }
    fclose(settings);

    // Меняем \n на \0
    if (file_path[strlen(file_path) - 1] == '\n')
        file_path[strlen(file_path) - 1] = '\0';

    if (!correct_name(file_path)) {
        strcpy(file_path, DIR_NAME "/" FILE_NAME);
        return 1; // Возвращаем путь DIR_NAME / FILE_NAME и код "путь не влез"
    }
    return 0;
}

// Изменить путь до файла, в который нужно напisyвать данные
int change_path() {
    /*
    Возвращает:
        -1: У пользователя нет прав даже для папки DIR_NAME
        0: Путь до файла успешно получен
        1: Отмена ввода
    */
    char old_file_path[FILE_NAME_LEN]; // Буфер для хранения пути до файла
    int old_exists = 0; // Путь до файла, указанный в settings.txt существует и
корректен
    FILE* old_file;

    // Пробуем прочитать файл

```

```

FILE* settings = fopen(DIR_NAME "/" CONF_NAME, "r");
if (settings == NULL) { // Если файла нет, то создаём его
    settings = fopen(DIR_NAME "/" CONF_NAME, "w");
    if (settings == NULL) return -1; // Если его нельзя создать, то
возвращаем -1
    else fclose(settings);
    settings = fopen(DIR_NAME "/" CONF_NAME, "r");
}
// Если файл не пустой, то выведем информацию о пути
if (fgets(old_file_path, FILE_NAME_LEN, settings) != NULL) {
    old_exists = correct_name(old_file_path);
    printf("Текущий путь до файла %s", old_file_path);
    if (!old_exists) printf(" является некорректным");
    printf("\n");
}
fclose(settings);

// Ввод нового адреса и его проверка
printf("Данные о полётах хранятся в файле table.csv.\n");
printf("Введите новый адрес файла, например %s. Максимальная длина %d
символов\n",
    DIR_NAME "/" FILE_NAME, FILE_NAME_LEN - 1);

int new_file_error = 1; // Некорректно указан новый адрес
char new_file_path[FILE_NAME_LEN]; // Буфер для хранения пути до файла
FILE* new_file; // Новый файл

while (new_file_error) {
    if (input(new_file_path, FILE_NAME_LEN) == 1) return 1; // Отмена
ввода
    if (!correct_name(new_file_path))
        printf("Некорректно указан новый адрес: названия файла нет в
строке или не в конце\n");
    else // Проверим, что файл уже существует или его можно создать
        if (new_file = fopen(new_file_path, "r")) {
            printf("УСПЕШНО: файл по указанному адресу существует\n");
            fclose(new_file);
            new_file_error = 0;
        }
        else
            if (new_file = fopen(new_file_path, "w")) {
                printf("УСПЕШНО: файл по указанному адресу
создан\n");
                fclose(new_file);
                new_file_error = 0;
            }
            else printf("Некорректно указан новый адрес\n");
    }

// Проверим, что файл по адресу в файле существует
if (old_exists) {
    old_file = fopen(old_file_path, "r");
    if (old_file == NULL) old_exists = 0;
    else fclose(old_file);
}
// Нужно ли перенести информацию из прошлого файла в новый
if (old_exists) {
    printf("\nНужно ли перенести информацию из прошлого файла в новый?\n");
    printf("0 - Нет\n");
    printf("1 - Да\n");
    int adding_res;
    if (!int_input(&adding_res) && adding_res == 1 && // Если пользователь
ввёл 1

```

```

        (old_file = fopen(old_file_path, "r")) != NULL) { // Если файл
существует (да-да, ещё одна проверка)
        char read_buffer[64]; // Буфер для переноса
        if ((new_file = fopen(new_file_path, "a")) == NULL) // Если не
получилось открыть новый файл
            printf("Произошла ошибка, информация не была
перенесена\n");
        else {
            while (fgets(read_buffer, 64, old_file) != NULL)
                fputs(read_buffer, new_file);
            fclose(new_file);
        }
        fclose(old_file);
    }
}
// Нужно ли удалить прошлый файл
if (old_exists) {
    printf("\nНужно ли удалить прошлый файл?\n");
    printf("0 - Нет\n");
    printf("1 - Да\n");
    int adding_res;
    if (!int_input(&adding_res) && adding_res == 1) {
        remove(old_file_path);
        printf("Файла больше не существует >:3\n");
    }
}
// Записываем адрес нового файла в settings.txt
settings = fopen(DIR_NAME "/" CONF_NAME, "w");
if (settings == NULL) return -1; // Если его нельзя создать, то возвращаем -
1
fputs(new_file_path, settings);
fclose(settings);
return 0;
}

// Записать одну строчку файла в структуру
int read_line(FILE *table, Flight *flight) {
    int days = 0; // Хранение дней в int, например [1, 2, 4] -> 1240000
    int res = fscanf(table, "%d;%15[^\n];%15[^\n];%d;%d;%d;%lf",
        &flight->fnum,
        flight->name,
        flight->dest,
        &days,
        &flight->dep_time,
        &flight->arr_time,
        &flight->price);
    for (int i = 6; i >= 0; i--) { // Переводим int days в массив
        flight->days[i] = days % 10;
        days /= 10;
    }
    return res;
}

// Записать одну структуру таблицы в файл
int write_line(FILE *table, Flight flight) {
    int days = 0; // Хранение дней в int, например [1, 2, 4] -> 1240000
    // Переводим массив в int days
    for (int i = 0; i < 7; i++) days = days * 10 + flight.days[i];
    int res = fprintf(table, "%d;%s;%s;%d;%d;%d;%lf\n",
        flight.fnum,
        flight.name,
        flight.dest,
        days,

```

```

        flight.dep_time,
        flight.arr_time,
        flight.price);
    return res;
}

// Посчитать количество подходящих под фильтр строк
int count_lines(FILE *table, Flight_filter filters[], int *len, int *fil_len) {
    *len = 0;           // Сколько записей получилось успешно прочитать
    *fil_len = 0;       // Сколько записей, подходящих фильтрам, получилось
    успешно прочитать
    /*
    После использования рекомендуется использовать rewind()
    Возвращает:
        0: все строки успешно прочитаны или файл пустой
        1: не все строки были прочитаны и файл не пустой
        -1: ни одна строка не была прочитана, но файл не пустой
    */

    Flight flight_buffer; // Буфер для чтения
    int col_num; // Сколько столбцов успешно прочитано

    while ((col_num = read_line(table, &flight_buffer)) == FIELDS_NUM) {
        *len = *len + 1;
        if (compare_flight(flight_buffer, filters))
            *fil_len = *fil_len + 1; // Только если подходит под фильтр
    }
    if (*len != 0)
        return (col_num != EOF);
    return -(col_num != EOF);
}

```

II.A.5 Код structure.h

```

#pragma once

#define TEXT_LEN 16          // Сколько памяти нужно выделить на хранение текстовых
                              полей
#define COLS_WIDE 15        // Ширина каждой колонки
#define FIELDS_NUM 7        // Количество полей структуры
#define FILTERS_NUM 2       // Сколько фильтров можно одновременно использовать

// Структура информации о полёте
typedef struct {
    int fnum;                // Номер рейса                (flight number)
    char name[TEXT_LEN];     // Тип самолёта        (название модели)
    char dest[TEXT_LEN];     // Пункт назначения    (destination)
    int days[7];             // Дни отправления
    int dep_time;            // Время вылета в минутах (departure time)
    int arr_time;            // Время прилёта в минутах (arrival time)
    double price;            // Цена билета        (десятичная дробь)
} Flight;

// Структура для фильтров записей
typedef struct {
    int fnum;                // Номер рейса                (flight number)
    char name[TEXT_LEN];     // Тип самолёта        (название модели)
    char dest[TEXT_LEN];     // Пункт назначения    (destination)
    int days[7];             // Дни отправления
    int dep_time;            // Время вылета в минутах (departure time)
    int arr_time;            // Время прилёта в минутах (arrival time)
    double price;            // Цена билета        (десятичная дробь)
}

```



```

        int apply[FIELDS_NUM];    // Какие столбцы нужно фильтровать (и каким образом)
        int logic[FIELDS_NUM];    // Какую операцию нужно применять при сложении
    фильтров
} Flight_filter;

// Ввод структуры
// Ввод поля fnum
int input_fnum(Flight *pointer);

// Ввод текстового поля
int input_text_field(char *field, const char *info);
// Ввод поля name
int input_name(Flight *pointer);
// Ввод поля dest
int input_dest(Flight *pointer);

// Ввод поля days
int input_days(Flight *pointer);

// Ввод временного поля
int input_time_field(int *field, const char *info);
// Ввод поля dep_time
int input_dep_time(Flight *pointer);
// Ввод поля arr_time
int input_arr_time(Flight *pointer);

// Ввод поля price
int input_price(Flight *pointer);

// Ввод полной структуры
int input_flight(Flight *pointer);

// Вывод структур
// Печать части таблицы без данных
void print_table_line(char vert = '+', char horiz = '-', int cols_num = FIELDS_NUM +
1, int cols_wide = COLS_WIDE);
// Печать шапки таблицы
void print_head();
// Печать "пустой строки"
void print_blank(int cols_num = FIELDS_NUM + 1, int cols_wide = COLS_WIDE);

// Вывод одной записи
void print_flight(Flight flight, int cols_wide = COLS_WIDE);
// Вывод всех записей
int print_flights(Flight_filter filters[], int *len, int *fil_len, int full_info =
0);

// Работа с фильтрами
// Соответствует ли запись всем заданным фильтрам
int compare_flight(Flight flight, Flight_filter filters[]);

// Вывод заданного поля фильтра
int print_filter(Flight_filter filter, int field, int is_first);

// Вывод меню с фильтрами
int print_filters(Flight_filter filters[]);

// Установка фильтра
int set_filter(Flight_filter *p_filter, int field, int is_first = 0);

```

```

// Удаление и изменение записей
// Открыть текущий файл table.csv для чтения и dable.csv для записи
int open_dable(char *file_path, char *new_file_path, FILE **new_table, FILE
**old_table);

// Изменение одной структуры
int edit_all_filtered(char *file_path, Flight_filter filters[]);
// Изменение всех записей по фильтру
int edit_from_array(char *file_path, int len, int *to_edit, int edit_len);

// Удалить все подходящие под фильтр записи
int delete_all_filtered(char *file_path, Flight_filter filters[]);
// Удалить все записи, чьи номера есть в массиве
int delete_from_array(char *file_path, int len, int *to_del, int del_len);

```

П.А.6 Код structure.cpp

```

#define _CRT_SECURE_NO_WARNINGS

#include <stdio.h> // Для FILE, printf, fopen, fclose, rewind, rename,
remove
#include <stdlib.h> // Для system, _itoa
#include <string.h> // Для strlen, strcpy, strchr, strstr
#include <math.h> // Для log10

#include "structure.h"
#include "file_funcs.h" // Для FILE_NAME_LEN, get_file, read_line,
write_line, count_lines
#include "input_funcs.h" // Для CANCEL, input, int_input, double_input,
number_array_input, input_time

// Стандартный strlwr не работает(
char* mystrlwr(char* string) {
    // Преобразовать буквы верхнего регистра строки string в буквы нижнего
    регистра
    // string: изменяемая строка
    for (int i = 0; string[i]; i++)
        if (('A' <= string[i]) && (string[i] <= 'Z')) string[i] += 'a' - 'A';
        else if (('A' <= string[i]) && (string[i] <= 'Я')) string[i] += 'a' -
        'A';
    return string;
}

// Ввод поля fnum
int input_fnum(Flight *pointer) {
    const char *info = "Введите номер рейса (>0)";
    printf("\n%s: ", info);
    int result; // Результат ввода: 0 - успешно, 1 - отмена
    do {
        result = int_input(&pointer->fnum);
        if (result > 2 || result == 0 && pointer->fnum < 1)
            printf("Некорректный ввод. %s: ", info);
    } while (result > 1 || (result == 0 && pointer->fnum < 1));
    return result;
}

// Ввод текстового поля
int input_text_field(char *field, const char *info) {
    printf("\n%s: ", info);
    char user_input[TEXT_LEN]; // Буфер для ввода

```

```

        int input_res;                                // Результат ввода: 0 - успешно, 1 -
отмена                                                // Сколько символов не влезло
        int overflow;

        do input_res = input(user_input, TEXT_LEN, &overflow);
        while (input_res == 2 ||                      // Пока пустая строка
                strchr(user_input, ';') != NULL);      // Или содержит ';'

        if (!input_res) {
            strcpy(field, user_input);
            if (overflow)
                printf("Строчка не влезла целиком. Сохранено: %s", user_input);
        }
        return input_res;
    }

    // Ввод поля name
    int input_name(Flight *pointer) {
        const char *info = "Введите тип самолёта (без знака \";\");";
        return input_text_field(pointer->name, info);
    }

    // Ввод поля dest
    int input_dest(Flight *pointer) {
        const char *info = "Введите пункт назначения (без знака \";\");";
        return input_text_field(pointer->dest, info);
    }

    // Ввод поля days
    int input_days(Flight *pointer) {
        const char *info = "Введите дни отправления (1-7) через запятую";
        printf("\n%s: ", info);
        int days[7]; // Массив номеров дней
        int last_day = 0; // Индекс последнего элемента массива
        int result = 0; // Результат ввода: 0 - успешно, 1 - отмена

        do {
            if (result == 3) printf("Некорректный ввод. %s: ", info);
            result = number_array_input(days, &last_day);
            if (result == 1) return 1;
        } while (result);

        // Запись в структуру
        for (int i = 0; i < last_day; i++)
            pointer->days[i] = days[i];
        // Всё остальное заполняем нулями
        for (int i = last_day; i < 7; i++)
            pointer->days[i] = 0;
        return 0;
    }

    // Ввод временного поля
    int input_time_field(int *field, const char *info) {
        printf("\n%s: ", info);
        int result; // Результат ввода: 0 - успешно, 1 - отмена
        do {
            result = input_time(field);
            if (result == 3) printf("Некорректный ввод. %s: ", info);
        } while (result > 1);
        return result;
    }

    // Ввод поля dep_time

```

```

int input_dep_time(Flight *pointer) {
    const char *info = "Введите время вылета (например, 13:00)";
    return input_time_field(&pointer->dep_time, info);
}

// Ввод поля arr_time
int input_arr_time(Flight *pointer) {
    const char *info = "Введите время прилёта (например, 14:00)";
    return input_time_field(&pointer->arr_time, info);
}

// Ввод поля price
int input_price(Flight *pointer) {
    const char* info = "Введите цену билета (>0)";
    printf("\n%s: ", info);
    int result; // Результат ввода: 0 - успешно, 1 - отмена
    do {
        result = double_input(&pointer->price);
        if (result > 2 || result == 0 && pointer->price <= 0)
            printf("Некорректный ввод. %s: ", info);
    } while (result > 1 || (result == 0 && (*pointer).price <= 0));
    return result;
}

// Ввод полной структуры
int input_flight(Flight *pointer) {
    printf("\nВведите запись по столбцам. "
        "Чтобы закончить ввод, введите \"%s\"", CANCEL);
    int result = 0; // Результат ввода: 0 - успешно, 1 - отмена
    // Ввод всех полей
    if (input_fnum(pointer) ||
        input_name(pointer) || input_dest(pointer) ||
        input_days(pointer) ||
        input_dep_time(pointer) || input_arr_time(pointer) ||
        input_price(pointer))
    {
        printf("\nВвод завершён, запись не была добавлена.\n");
        result = 1;
    }
    else printf("\nВвод завершён, запись добавлена в таблицу.\n");
    return result;
}

// Печать части таблицы без данных
void print_table_line(char vert, char horiz, int cols_num, int cols_wide) {
    /*
    vert: разделитель колонок
    horiz: разделитель строк (если не стоит vert)
    cols_num: количество столбцов
    cols_wide: длина столбца
    */
    for (int i = 0; i <= cols_num * (cols_wide + 1); i++)
        i % (cols_wide + 1) ? printf("%c", horiz) : printf("%c", vert);
    printf("\n");
}

// Печать шапки таблицы
void print_head() {
    printf("Список авиарейсов:\n");
    print_table_line();
    // Первая строка
    printf("|      Номер      |      Номер      |      Тип      |      Пункт      "

```

```

        "|      Дни      |      Время      |      Время      |      Цена
|\n");
    // Вторая строка
    printf("|      в таблице      |      рейса      |      самолёта      |      назначения      "
        "|      отправления      |      вылета      |      прилёта      |      билета
|\n");
    print_table_line();
}

// Печать "пустой строки"
void print_blank(int cols_num, int cols_wide) {
    for (int col = 0; col < cols_num; col++) {
        printf("|");
        for (int i = 1; i < (cols_wide + 1); i++)
            if ((cols_wide) / 2 <= i || i <= (cols_wide + 1) / 2 + 1)
                printf(".");
            else printf(" ");
    }
    printf("|\n");
    print_table_line();
}

// Вывод одной записи
void print_flight(Flight flight, int cols_wide) {
    // Получим длины каждого поля
    int len_fnum = 1;
    if (flight.fnum)
        len_fnum = log10(flight.fnum) + 1;                                // fnum

    int len_name = strlen(flight.name);                                    // name
    int len_dest = strlen(flight.dest);                                    // dest
    int len_dep_time = (flight.dep_time / 60 >= 10) + 1;                  // dep_time
    int len_arr_time = (flight.arr_time / 60 >= 10) + 1;                  // arr_time

    int len_price = 4;
    if (flight.price > 0)
        len_price = log10(flight.price) + 1 + 3;                        // price

    int len_days = 0;
    for (int i = 0; flight.days[i] && i < 7; i++)
        len_days += 2;
    len_days -= 1;
    // days

    // Теперь выведем каждое поле
    int left_space; // Сколько пробелов нужно добавить слева
    // Номер рейса
    left_space = (cols_wide - len_fnum) / 2;
    printf("|"); for (int i = 0; i < left_space; i++) printf(" ");
    printf("%d", flight.fnum);
    for (int i = 0; i < cols_wide - len_fnum - left_space; i++) printf(" ");

    // Тип самолёта
    left_space = (cols_wide - len_name) / 2;
    printf("|"); for (int i = 0; i < left_space; i++) printf(" ");
    for (int i = 0; flight.name[i] && i < cols_wide; i++) printf("%c",
flight.name[i]);
    for (int i = 0; i < cols_wide - len_name - left_space; i++) printf(" ");

    // Пункт назначения
    left_space = (cols_wide - len_dest) / 2;
    printf("|"); for (int i = 0; i < left_space; i++) printf(" ");

```

```

    for (int i = 0; flight.dest[i] && i < cols_wide; i++) printf("%c",
flight.dest[i]);
    for (int i = 0; i < cols_wide - len_dest - left_space; i++) printf(" ");

    // Дни отправления
    left_space = (cols_wide - len_days) / 2;
    printf("|%*c", cols_wide / 2 - left_space; i++) printf(" ");
    printf("%d", flight.days[0]);
    for (int i = 1; flight.days[i] && i < 7; i++) printf(",%d", flight.days[i]);
    for (int i = 0; i < cols_wide - len_days - left_space; i++) printf(" ");

    // Время вылета
    printf("|%*c", cols_wide / 2 - len_dep_time, ' ');
    printf("%d:%.2d", flight.dep_time / 60, flight.dep_time % 60);
    printf("%*c", cols_wide / 2 - 2, ' ');

    // Время прилёта
    printf("|%*c", cols_wide / 2 - len_arr_time, ' ');
    printf("%d:%.2d", flight.arr_time / 60, flight.arr_time % 60);
    printf("%*c", cols_wide / 2 - 2, ' ');

    // Цена билета
    if (len_price <= 13) {
        left_space = (cols_wide - len_price) / 2;
        printf("|%*c", left_space, ' ');
        printf("%.2lf", flight.price);
        printf("%*c", cols_wide - len_price - left_space, ' ');
    }
    else printf("|%9.9e", flight.price);
    printf("\n");
}

// Вывод всех записей
int print_flights(Flight_filter filters[], int *len, int *fil_len, int full_info) {
    /*
    Возвращает:
        -3: Файл повреждён и не может быть прочитан
        -2: Некорректный путь до файла
        -1: У пользователя нет прав даже для папки DIR_NAME
        0: Файл успешно прочитан
        1: Файл повреждён, но строки прочитаны
    */
    char file_path[FILE_NAME_LEN];
    int path_res = get_file(file_path, FILE_NAME_LEN);
    if (path_res == -1) return -1; // Если всё плохо, то завершаем работу с
кодом -1
    if (path_res == 1) { // Путь до файла в settings.txt указан некорректно
        printf("Некорректный путь до файла\n");
        return -2;
    }
    FILE* table = fopen(file_path, "r");
    if (table == NULL) { // Если файла нет, завершаем работу
        printf("Некорректный путь до файла\n");
        return -2;
    }
    int count_res = count_lines(table, filters, len, fil_len);

    // Вывод информации
    if (full_info) {
        // Проверка на ошибки чтения
        if (count_res == -1) {
            printf("Файл повреждён и не может быть прочитан\n");
            fclose(table);
        }
    }
}

```

```

        return -3;
    }
    else if (count_res == 1)
        printf("Файл был повреждён. Прочитано только %d строк\n", *len);
    // Проверка на кол-во записей в таблице
    if (*len == 0) {
        printf("Таблица пустая\n");
        fclose(table);
        return 0;
    }
    else if (*fil_len == 0) {
        printf("Нет записей, подходящих данным фильтрам\n");
        fclose(table);
        return 0;
    }
}
else if (*len) printf("Авиарейсы Mark company:\n");
rewind(table);

// Вывод записей
Flight flight_buffer; // Буфер для чтения
if (*fil_len) print_head();
if (*fil_len < 6 || full_info) { // Выведем все записи, если их <6 или нужно
вывести всё
    for (int line = 0; line < *len; line++) {
        read_line(table, &flight_buffer);
        if (!compare_flight(flight_buffer, filters)) continue;
        // Номер в таблице
        int len_line = log10(line + 1) + 1;
        int left_space = (COLS_WIDE - len_line) / 2;
        printf("|%*c", left_space, ' ');
        printf("%d", line + 1);
        printf("%*c", COLS_WIDE - len_line - left_space, ' ');
        // Сама запись
        print_flight(flight_buffer);
        print_table_line();
    }
}
else { // Иначе выводим первые 2 и последние 2
    Flight four_flights[4]; // Массив для 4-ёх записей
    int four_numbers[4]; // Массив для нужных номеров
    int counter = 0; // Сколько строчек вывели
    int i; // Для перебора строчек
    for (i = 0; i <= *len && counter < 2; i++) {
        read_line(table, &flight_buffer);
        if (compare_flight(flight_buffer, filters)) {
            four_flights[counter] = flight_buffer;
            four_numbers[counter++] = i + 1;
        }
    }
    four_flights[3] = four_flights[1];
    four_numbers[3] = four_numbers[1];
    for (; i < *len; i++) {
        read_line(table, &flight_buffer);
        if (compare_flight(flight_buffer, filters)) {
            four_flights[2] = four_flights[3];
            four_flights[3] = flight_buffer;
            four_numbers[2] = four_numbers[3];
            four_numbers[3] = i + 1;
        }
    }
}
// Вывод записей
for (int i = 0; i < 4; i++) {

```

```

        if (i == 2) print_blank();
        // Номер в таблице
        int len_num = log10(four_numbers[i]) + 1;
        int left_space = (COLS_WIDE - len_num) / 2;
        printf("|%*c", left_space, ' ');
        printf("%d", four_numbers[i]);
        printf("%*c", COLS_WIDE - len_num - left_space, ' ');
        // Сама запись
        print_flight(four_flights[i]);
        print_table_line();
    }
}
fclose(table);
return count_res;
}

// Соответствует ли запись всем заданным фильтрам
int compare_flight(Flight flight, Flight_filter filters[]) {
    int is_good = 1; // Запись изначально подходит
    for (int i = 0; i < FIELDS_NUM && is_good; i++)
        for (int fil = 0; fil < FILTERS_NUM; fil++) {
            Flight_filter cur_filter = filters[fil]; // Текущий фильтр
            int cur_good = 1; // Подходит ли текущее поле под текущий
фильтр
            if (i == 0 && cur_filter.apply[0]) { // Поле fnum
                char flight_fnum[16], filter_fnum[16];
                _itoa(flight.fnum, flight_fnum, 10);
                _itoa(cur_filter.fnum, filter_fnum, 10);
                if (strstr(flight_fnum, filter_fnum) == NULL) cur_good =
0;
            }
            if (i == 1 && cur_filter.apply[1]) { // Поле name
                char flight_name[16], filter_name[16];
                mystrlwr(strcpy(flight_name, flight.name));
                mystrlwr(strcpy(filter_name, cur_filter.name));
                if (strstr(flight_name, filter_name) == NULL) cur_good =
0;
            }
            if (i == 2 && cur_filter.apply[2]) { // Поле dest
                char flight_dest[16], filter_dest[16];
                mystrlwr(strcpy(flight_dest, flight.dest));
                mystrlwr(strcpy(filter_dest, cur_filter.dest));
                if (strstr(flight_dest, filter_dest) == NULL) cur_good =
0;
            }
            if (i == 3 && cur_filter.apply[3]) { // Поле days
                int common = 0;
                for (int i = 0; i < 7 && cur_filter.days[i]; i++)
                    for (int j = 0; j < 7 && flight.days[j]; j++)
                        if (cur_filter.days[i] == flight.days[j])
common += 1;
                if (!common) cur_good = 0;
            }
            if (i == 4 && cur_filter.apply[4]) { // Поле dep_time
                if (cur_filter.apply[4] == 1) // flight > filter
                    if (flight.dep_time <= cur_filter.dep_time) cur_good
= 0;
                if (cur_filter.apply[4] == 2) // flight < filter
                    if (flight.dep_time >= cur_filter.dep_time) cur_good
= 0;
                if (cur_filter.apply[4] == 3) // flight = filter

```



```

        if (flight.dep_time != cur_filter.dep_time) cur_good
= 0;
        if (cur_filter.apply[4] == 4) // flight >= filter
            if (flight.dep_time < cur_filter.dep_time) cur_good
= 0;
        if (cur_filter.apply[4] == 5) // flight <= filter
            if (flight.dep_time > cur_filter.dep_time) cur_good
= 0;
    }
    if (i == 5 && cur_filter.apply[5]) { // None arr_time
        if (cur_filter.apply[5] == 1) // flight > filter
            if (flight.arr_time <= cur_filter.arr_time) cur_good
= 0;
        if (cur_filter.apply[5] == 2) // flight < filter
            if (flight.arr_time >= cur_filter.arr_time) cur_good
= 0;
        if (cur_filter.apply[5] == 3) // flight = filter
            if (flight.arr_time != cur_filter.arr_time) cur_good
= 0;
        if (cur_filter.apply[5] == 4) // flight >= filter
            if (flight.arr_time < cur_filter.arr_time) cur_good
= 0;
        if (cur_filter.apply[5] == 5) // flight <= filter
            if (flight.arr_time > cur_filter.arr_time) cur_good
= 0;
    }
    if (i == 6 && cur_filter.apply[6]) { // None price
        if (cur_filter.apply[6] == 1) // flight > filter
            if (flight.price <= cur_filter.price) cur_good = 0;
        if (cur_filter.apply[6] == 2) // flight < filter
            if (flight.price >= cur_filter.price) cur_good = 0;
        if (cur_filter.apply[6] == 3) // flight = filter
            if (flight.price != cur_filter.price) cur_good = 0;
        if (cur_filter.apply[6] == 4) // flight >= filter
            if (flight.price < cur_filter.price) cur_good = 0;
        if (cur_filter.apply[6] == 5) // flight <= filter
            if (flight.price > cur_filter.price) cur_good = 0;
    }
    // Применяем логическую операцию
    if (cur_filter.logic[i] == 0) is_good = is_good &&
cur_good;
    else if (cur_filter.logic[i] == 1) is_good = is_good ||
cur_good;
    }
    return is_good;
}

// Вывод заданного поля фильтра
int print_filter(Flight_filter filter, int field, int is_first) {
    if (!filter.apply[field]) return 0; // Если не нужно применять фильтр
    if (!is_first) // Если он не стоит первым
        if (filter.logic[field] == 0) printf(" && ");
        else if (filter.logic[field] == 1) printf(" || ");
    // Выведем сам фильтр
    const char apply_type[5][3] = { ">", "<", "=", ">=", "<=" };
    if (field == 0) printf("%d", filter.fnum);
    else if (field == 1) printf("%s", filter.name);
    else if (field == 2) printf("%s", filter.dest);
    else if (field == 3) {
        printf("%d", filter.days[0]);
        for (int i = 1; i < 7 && filter.days[i]; i++)
            printf(", %d", filter.days[i]);
    }
}

```

```

        else if (field == 4)
            printf("%s%d:%.2d", apply_type[filter.apply[4] - 1], filter.dep_time /
60, filter.dep_time % 60);
        else if (field == 5)
            printf("%s%d:%.2d", apply_type[filter.apply[5] - 1], filter.arr_time /
60, filter.arr_time % 60);
        else if (field == 6)
            printf("%s%.2lf", apply_type[filter.apply[6] - 1], filter.price);
        return 1;
    }

// Вывод меню с фильтрами
int print_filters(Flight_filter filters[]) {
    system("cls");
    int len, fil_len;
    if (print_flights(filters, &len, &fil_len, 1) < 0 || len == 0) return 0;
    printf("\n");
    const char *cols_names[FIELDS_NUM]; // Названия колонок
    cols_names[0] = "Номер рейса ";
    cols_names[1] = "Тип самолёта ";
    cols_names[2] = "Пункт назначения";
    cols_names[3] = "Дни отправления ";
    cols_names[4] = "Время вылета ";
    cols_names[5] = "Время прилёта ";
    cols_names[6] = "Цена билета ";
    printf("Текущие фильтры:");
    for (int col = 0; col < FIELDS_NUM; col++) {
        int counter = 0; // Сколько фильтров вывели
        printf("\n%d. %s - ", col + 1, cols_names[col]);
        for (int i = 0; i < FILTERS_NUM; i++)
            counter += print_filter(filters[i], col, counter == 0);
        if (counter == 0) printf("не используется");
    }
    return 1;
}

// Установка фильтра
int set_filter(Flight_filter *p_filter, int field, int is_first) {
    int filter_result = 0; // Результат ввода фильтра
    int apply_result = 1; // Выбранный оператор сравнения (для некоторых
полей)
    int logic_result = 0; // Выбранный логический оператор (если не первый =
!is_first)
    Flight holder; // Буфер ввода
    if (field == 0) filter_result = input_fnum(&holder); // Поле
fnum
    else if (field == 1) filter_result = input_name(&holder); // Поле name
    else if (field == 2) filter_result = input_dest(&holder); // Поле dest
    else if (field == 3) filter_result = input_days(&holder); // Поле days
    else if (field == 4) filter_result = input_dep_time(&holder); // Поле
dep_time
    else if (field == 5) filter_result = input_arr_time(&holder); // Поле
arr_time
    else if (field == 6) filter_result = input_price(&holder); // Поле
price

    // Выбор оператора сравнения
    if (!filter_result && 4 <= field && field <= 6) {
        if (field == 6) { // Цена
            printf("\nВведите оператор сравнения:");
            printf("\n1. \">\" (Дороже %.2lf)", holder.price);
            printf("\n2. \"<\" (Дешевле %.2lf)", holder.price);

```

```

        printf("\n3. \"=\" (Ровно %.2lf)", holder.price);
        printf("\n4. \">=\" (Дороже или ровно %.2lf)", holder.price);
        printf("\n5. \"<=\" (Дешевле или ровно %.2lf)\n", holder.price);
    }
    else { // Время
        int hours, minutes;
        if (field == 4) { // dep_time
            hours = holder.dep_time / 60;
            minutes = holder.dep_time % 60;
        }
        if (field == 5) { // arr_time
            hours = holder.arr_time / 60;
            minutes = holder.arr_time % 60;
        }
        printf("\nВведите оператор сравнения:");
        printf("\n1. \">\" (Позже %d:%.2d)", hours, minutes);
        printf("\n2. \"<\" (Раньше %d:%.2d)", hours, minutes);
        printf("\n3. \"=\" (Точно в %d:%.2d)", hours, minutes);
        printf("\n4. \">=\" (Позже или в %d:%.2d)", hours, minutes);
        printf("\n5. \"<=\" (Раньше или в %d:%.2d)\n", hours, minutes);
    }
    int answer_res; // Результат ввода оператора
    do answer_res = int_input(&apply_result);
    while (answer_res > 1 || (answer_res == 0 && (apply_result < 1 || 5 <
apply_result)));
}

// Выбор логического оператора
if (!filter_result && !is_first) {
    printf("\nВведите логический оператор:");
    printf("\n0. Логическое И (&&)");
    printf("\n1. Логическое ИЛИ (||)\n");
    do filter_result = int_input(&logic_result);
    while (filter_result > 1 || (logic_result != 0 && logic_result != 1));
}

// Сохраняем фильтр, если до этого всё прошло успешно
if (!filter_result) {
    if (field == 0) p_filter->fnum = holder.fnum;
    else if (field == 1) strcpy(p_filter->name, holder.name);
    else if (field == 2) strcpy(p_filter->dest, holder.dest);
    else if (field == 3) for (int i = 0; i < 7; i++) p_filter->days[i] =
holder.days[i];
    else if (field == 4) p_filter->dep_time = holder.dep_time;
    else if (field == 5) p_filter->arr_time = holder.arr_time;
    else if (field == 6) p_filter->price = holder.price;
    p_filter->apply[field] = apply_result;
    p_filter->logic[field] = logic_result;
    printf("\nФильтр успешно добавлен.");
}
else printf("\nФильтр не был добавлен");
return filter_result;
}

// Открыть текущий файл table.csv для чтения и dable.csv для записи
int open_dable(char *file_path, char *new_file_path, FILE **new_table, FILE
**old_table) {
    /*
    Возвращает:
        -1: Недостаток прав
        0: Успешное открытие
    */

```

```

        *strstr(strcpy(new_file_path, file_path), "table") = 'd'; // Тот же путь,
        только ../dable.csv
        // Открываем dable.csv для записи
        *new_table = fopen(new_file_path, "w");
        if (new_table == NULL) return -1;
        // Открываем текущий файл table.csv для чтения
        *old_table = fopen(file_path, "r");
        if (old_table == NULL) {
            fclose(*new_table);
            return -1;
        }
        return 0;
    }
}

```

```

// Изменение всех записей по фильтру
int edit_all_filtered(char *file_path, Flight_filter filters[]) {
    /*
    Возвращает:
        -2: Ошибка чтения
        -1: Недостаток прав
        0: Успешное изменение
        1: Отмена изменения
    */
    char new_file_path[FILE_NAME_LEN];
    FILE *new_table, *old_table;
    if (open_dable(file_path, new_file_path, &new_table, &old_table) == -1)
        return -1;

```

```

        const char* info = "Введите номера столбцов, которые вы хотите изменить,
        через запятую";
        printf("\n%s:\n", info);
        const char* cols_names[FIELDS_NUM]; // Названия колонок
        cols_names[0] = "Номер рейса";
        cols_names[1] = "Тип самолёта";
        cols_names[2] = "Пункт назначения";
        cols_names[3] = "Дни отправления";
        cols_names[4] = "Время вылета";
        cols_names[5] = "Время прилёта";
        cols_names[6] = "Цена билета";
        for (int col = 0; col < FIELDS_NUM; col++)
            printf("%d - %s\n", col + 1, cols_names[col]);
        printf("Ваш выбор: ");

```

```

        // Какие столбцы нужно поменять
        int cols[FIELDS_NUM]; // Массив номеров столбцов
        int last_ind = 0; // Индекс последнего элемента массива
        int cols_res = 0; // Результат ввода массива столбцов
        do {
            if (cols_res == 3) printf("Некорректный ввод.\n%s: ", info);
            cols_res = number_array_input(cols, &last_ind);
        } while (cols_res > 1);
        if (cols_res == 1) return 1; // Отмена изменения

```

```

        // Создаём шаблон изменения
        int change_res = 0; // Результат ввода новых значений
        Flight new_data; // Тут будем хранить введённые данные
        for (int i = 0; i < last_ind && !change_res; i++)
            if (cols[i] == 1) change_res = input_fnum(&new_data);
            else if (cols[i] == 2) change_res = input_name(&new_data);
            else if (cols[i] == 3) change_res = input_dest(&new_data);
            else if (cols[i] == 4) change_res = input_days(&new_data);
            else if (cols[i] == 5) change_res = input_dep_time(&new_data);

```

```

        else if (cols[i] == 6) change_res = input_arr_time(&new_data);
        else if (cols[i] == 7) change_res = input_price(&new_data);

// Если строка не подходит под фильтр, то переписываем её
// Иначе изменяем её по шаблону и записываем
int is_error = 0; // Ошибка чтения
Flight buffer; // Буфер для чтения полёта
int len, fil_len; // Количество записей в файле
count_lines(old_table, filters, &len, &fil_len);
rewind(old_table);

for (int cur_line = 0; cur_line < len && !is_error; cur_line++)
    if (read_line(old_table, &buffer) != FIELDS_NUM) is_error = 1;
    else if (!compare_flight(buffer, filters)) write_line(new_table,
buffer);
        else {
            for (int i = 0; i < last_ind && !change_res; i++)
                if (cols[i] == 1) buffer.fnum = new_data.fnum;
                else if (cols[i] == 2) strcpy(buffer.name, new_data.name);
                else if (cols[i] == 3) strcpy(buffer.dest, new_data.dest);
                else if (cols[i] == 4) for (int j = 0; j < 7; j++)
buffer.days[j] = new_data.days[j];
                else if (cols[i] == 5) buffer.dep_time =
new_data.dep_time;
                else if (cols[i] == 6) buffer.arr_time =
new_data.arr_time;
                else if (cols[i] == 7) buffer.price = new_data.price;
                write_line(new_table, buffer);
            }

// Удаляем прошлый файл и переименовываем новый
fclose(new_table);
fclose(old_table);
remove(file_path);
rename(new_file_path, file_path);

if (is_error) return -2;
return change_res;
}

// Изменить все записи, чьи номера есть в массиве
int edit_from_array(char *file_path, int len, int *to_edit, int edit_len) {
/*
Возвращает:
-2: Ошибка чтения
-1: Недостаток прав
0: Успешное изменение
1: Отмена изменения
*/
char new_file_path[FILE_NAME_LEN];
FILE* new_table, * old_table;
if (open_dable(file_path, new_file_path, &new_table, &old_table) == -1)
return -1;

const char* info = "Введите номера столбцов, которые вы хотите изменить,
через запятую";
printf("\n%s:\n", info);
const char* cols_names[FIELDS_NUM]; // Названия колонок
cols_names[0] = "Номер рейса";
cols_names[1] = "Тип самолёта";
cols_names[2] = "Пункт назначения";
cols_names[3] = "Дни отправления";
cols_names[4] = "Время вылета";

```

```

cols_names[5] = "Время прилёта";
cols_names[6] = "Цена билета";
for (int col = 0; col < FIELDS_NUM; col++)
    printf("%d - %s\n", col + 1, cols_names[col]);
printf("Ваш выбор: ");

// Какие столбцы нужно поменять
int cols[FIELDS_NUM]; // Массив номеров столбцов
int last_ind = 0; // Индекс последнего элемента массива
int cols_res = 0; // Результат ввода массива столбцов
do {
    if (cols_res == 3) printf("Некорректный ввод.\n%s: ", info);
    cols_res = number_array_input(cols, &last_ind);
} while (cols_res > 1);
if (cols_res == 1) return 1; // Отмена изменения

// Создаём шаблон изменения
int change_res = 0; // Результат ввода новых значений
Flight new_data; // Тут будем хранить введённые данные
for (int i = 0; i < last_ind && !change_res; i++)
    if (cols[i] == 1) change_res = input_fnum(&new_data);
    else if (cols[i] == 2) change_res = input_name(&new_data);
    else if (cols[i] == 3) change_res = input_dest(&new_data);
    else if (cols[i] == 4) change_res = input_days(&new_data);
    else if (cols[i] == 5) change_res = input_dep_time(&new_data);
    else if (cols[i] == 6) change_res = input_arr_time(&new_data);
    else if (cols[i] == 7) change_res = input_price(&new_data);

// Изменяем нужные строчки
int is_error = 0; // Ошибка чтения
Flight buffer; // Буфер для чтения полёта
int last_edit = 0; // Для прохода по массиву to_edit
int cur_line = 0; // Для прохода по файлу

printf("\n");
for (; cur_line < len && last_edit < edit_len && !is_error; cur_line++)
    if (read_line(old_table, &buffer) != FIELDS_NUM) is_error = 1;
    else if (cur_line + 1 < to_edit[last_edit]) write_line(new_table,
buffer);
    else {
        for (int i = 0; i < last_ind && !change_res; i++)
            if (cols[i] == 1) buffer.fnum = new_data.fnum;
            else if (cols[i] == 2) strcpy(buffer.name, new_data.name);
            else if (cols[i] == 3) strcpy(buffer.dest, new_data.dest);
            else if (cols[i] == 4) for (int j = 0; j < 7; j++)
buffer.days[j] = new_data.days[j];
            else if (cols[i] == 5) buffer.dep_time =
new_data.dep_time;
            else if (cols[i] == 6) buffer.arr_time =
new_data.arr_time;
            else if (cols[i] == 7) buffer.price = new_data.price;
        write_line(new_table, buffer);
        printf("Строка %d успешно изменена\n", cur_line + 1);
        last_edit++;
    }
for (; cur_line < len && !is_error; cur_line++)
    if (read_line(old_table, &buffer) != FIELDS_NUM) is_error = 1;
    else write_line(new_table, buffer);

// Удаляем прошлый файл и переименовываем новый
fclose(new_table);
fclose(old_table);
remove(file_path);

```

```

        rename(new_file_path, file_path);

        if (is_error) return -2;
        return change_res;
    }

    // Удалить все подходящие под фильтр записи
    int delete_all_filtered(char *file_path, Flight_filter filters[]) {
        /*
        Возвращает:
            -2: Ошибка чтения
            -1: Недостаток прав
            0: Успешное изменение
        */
        char new_file_path[FILE_NAME_LEN];
        FILE* new_table, * old_table;
        if (open_dable(file_path, new_file_path, &new_table, &old_table) == -1)
            return -1;

        // Если строка не подходит под фильтр, то переписываем её
        int is_error = 0; // Ошибка чтения
        Flight buffer; // Буфер для чтения полёта
        int len, fil_len; // Количество записей в файле
        count_lines(old_table, filters, &len, &fil_len);
        rewind(old_table);

        for (int cur_line = 0; cur_line < len && !is_error; cur_line++)
            if (read_line(old_table, &buffer) != FIELDS_NUM) is_error = 1;
            else if (!compare_flight(buffer, filters)) write_line(new_table,
buffer);

        // Удаляем прошлый файл и переименовываем новый
        fclose(new_table);
        fclose(old_table);
        remove(file_path);
        rename(new_file_path, file_path);

        if (is_error) return -2;
        return 0;
    }

    // Удалить все записи, чьи номера есть в массиве
    int delete_from_array(char *file_path, int len, int *to_del, int del_len) {
        /*
        Возвращает:
            -2: Ошибка чтения
            -1: Недостаток прав
            0: Успешное изменение
        */
        char new_file_path[FILE_NAME_LEN];
        FILE* new_table, * old_table;
        if (open_dable(file_path, new_file_path, &new_table, &old_table) == -1)
            return -1;

        // Удаление будем совершать следующим образом:
        // Если номер строки в массиве – пропускаем её
        int is_error = 0; // Ошибка чтения
        Flight buffer; // Буфер для чтения полёта
        int last_del = 0; // Для прохода по массиву to_del
        int cur_line = 0; // Для прохода по файлу

        printf("\n");
        for (; cur_line < len && last_del < del_len && !is_error; cur_line++)

```

```

        if (read_line(old_table, &buffer) != FIELDS_NUM) is_error = 1;
        else if (cur_line + 1 < to_del[last_del]) write_line(new_table,
buffer);
        else { printf("Строка %d успешно удалена\n", cur_line + 1); last_del++;
}
    for (; cur_line < len && !is_error; cur_line++)
        if (read_line(old_table, &buffer) != FIELDS_NUM) is_error = 1;
        else write_line(new_table, buffer);

    // Удаляем прошлый файл и переименовываем новый
    fclose(new_table);
    fclose(old_table);
    remove(file_path);
    rename(new_file_path, file_path);

    if (is_error) return -2;
    return 0;
}

```

П.А.7 Код functions.h

```

#pragma once

#include "structure.h"    // Для Flight_filter

// Подготовка пользователя
int user_prepare(char *file_path);

// Выбор первичного действия
int get_first_action(Flight_filter filters[]);

// Выбор действия по вводу и корректировке списка
int get_edit_action(Flight_filter filters[]);

// Меню "Изменение записей"
int edit_menu(char *file_path, Flight_filter filters[]);

// Меню "Удаление записей"
int delete_menu(char *file_path, Flight_filter filters[]);

// Меню "Установка фильтров"
int filters_menu(Flight_filter filters[]);

```

П.А.8 Код functions.cpp

```

#define _CRT_SECURE_NO_WARNINGS

#include <stdio.h>          // Для FILE, fopen, fclose, fputs, printf
#include <conio.h>          // Для _getch
#include <windows.h>        // Для system, SetConsoleCP и SetConsoleOutputCP
#include <direct.h>         // Для _mkdir

#include "functions.h"
#include "structure.h"      // Для Flight_filter, set_filter
#include "input_funcs.h"    // Для int_input
#include "file_funcs.h"     // Для DIR_NAME, FILE_NAME, CONF_NAME, change_path,
get_file

// Подготовка пользователя
int user_prepare(char *file_path) {
    /*

```



```

        Возвращает:
        -1: У пользователя нет прав даже для папки DIR_NAME
        0: Путь до файла успешно получен
        1: Слишком длинный путь (кто-то вручную поменял файл) – запись
        произойдёт по пути DIR_NAME/FILE_NAME
    */
    system("mode con cols=140 lines=40");
    SetConsoleCP(1251); SetConsoleOutputCP(1251);

    printf("Mark Company Flights\n");
    int dir_res = _mkdir(DIR_NAME); // Попробуем создать папку на случай, если её
    нету
    if (!dir_res) { // Если папки не было, то это новый пользователь
        printf("Данные о полётах хранятся в файле table.csv.\n");
        printf("Путь по умолчанию: \"%s\". Хотите поменять путь? (вы сможете
        изменить его позже)\n", DIR_NAME "/" FILE_NAME);
        printf("0 - Нет\n");
        printf("1 - Да\n");
        int answer; // Ответ на вопрос
        int use_default_path = 1; // Флаг "использовать стандартный путь"
        if (!int_input(&answer) && answer == 1)
            if ((use_default_path = change_path()) == -1)
                printf("Ошибка получения прав доступа к файлу %s\n",
                DIR_NAME "/" FILE_NAME);
        if (use_default_path != 0) {
            // Сохраняем стандартный путь
            FILE* settings = fopen(DIR_NAME "/" CONF_NAME, "w");
            if (settings == NULL) printf("Ошибка получения прав доступа к
            файлу %s\n", DIR_NAME "/" CONF_NAME);
            else {
                fputs(DIR_NAME "/" FILE_NAME, settings);
                fclose(settings);
            }
            // Сохраняем создаём файл, если его не существует
            FILE* flights = fopen(DIR_NAME "/" FILE_NAME, "r");
            if (flights == NULL) {
                flights = fopen(DIR_NAME "/" FILE_NAME, "w");
                if (flights == NULL) printf("Ошибка получения прав доступа
                к файлу %s\n", DIR_NAME "/" FILE_NAME);
            }
            if (flights != NULL) fclose(flights);
        }
        else printf("Убедитесь, что папка %s используется только программой.\n",
        DIR_NAME);
        printf("Нажмите любую кнопку, чтобы продолжить:");
        _getch();
        system("cls");
        int get_res;
        if ((get_res = get_file(file_path)) == -1)
            printf("Ошибка получения прав доступа к файлу %s\n", DIR_NAME "/"
            CONF_NAME);
        return get_res;
    }

    // Выбор первичного действия
    int get_first_action(Flight_filter filters[]) {
        // Ввод пока не получим число от 1 до 5
        int action; // Выбор пользователя
        do {
            system("cls");
            int len, fil_len;
            print_flights(filters, &len, &fil_len);

```

```

        if (fil_len) printf("\n");
        printf("Выберите действие:\n");
        printf("1 - Вывод таблицы\n");
        printf("2 - Ввод и корректировка списка\n");
        printf("3 - Установка фильтров\n");
        printf("4 - Изменить путь до файла\n");
        printf("5 - Выход из программы\n");
        printf("Ваш выбор: ");
    } while (int_input(&action) || (action < 1 || 5 < action));
    return action;
}

// Выбор действия по вводу и корректировке списка
int get_edit_action(Flight_filter filters[]) {
    // Ввод пока не получим число от 1 до 4
    int action; // Выбор пользователя
    do {
        system("cls");
        int len, fil_len;
        print_flights(filters, &len, &fil_len);
        if (fil_len) printf("\n");
        printf("Выберите действие по вводу и корректировке:\n");
        printf("1 - Ввод таблицы\n");
        printf("2 - Изменение записей\n");
        printf("3 - Удаление записей\n");
        printf("4 - Вернуться в главное меню\n");
        printf("Ваш выбор: ");
    } while (int_input(&action) || (action < 1 || 4 < action));
    return action;
}

// Меню "Изменение записей"
int edit_menu(char *file_path, Flight_filter filters[]) {
    system("cls");
    int len, fil_len;
    print_flights(filters, &len, &fil_len, 1);
    if (fil_len == 0) return 0;

    printf("\nВведите 0, чтобы изменить все вышепоказанные записи.");
    printf("\nВведите номера записей (1-%d), которые хотите изменить, через запятую: ", len);
    int* to_edit; // Массив индексов на удаление
    int res; // Результат ввода
    res = int_array_input(&to_edit, len);

    // Обработка ошибок
    if (res == -1) printf("\nОшибка выделения памяти.");
    else if (res == 1) printf("\nИзменение записей отменено.");
    else if (res == 2 || res == 3) printf("\nНекорректный ввод.");

    else if (to_edit[0] == 0) { // Если это "полный массив"
        free(to_edit);
        printf("\nВыбрано изменение всех записей по фильтру\n");
        int edit_result = edit_all_filtered(file_path, filters);
        if (edit_result == -2) printf("\nПроизошла ошибка чтения. Не все строки были изменены");
        else if (edit_result == -1) printf("\nИзменение отменено по причине недостатка прав");
        else if (edit_result == 0) printf("\nИзменение завершено");
        else if (edit_result == 1) printf("\nИзменение отменено");
    }
    else { // Просим у пользователя подтверждение операции и заодно считаем кол-во элементов

```

```

        int edit_len;
        printf("\nДля изменения выбраны следующие записи:\n%d", to_edit[0]);
        for (edit_len = 1; to_edit[edit_len]; edit_len++) printf(", %d",
to_edit[edit_len]);

        // Изменяем все указанные записи
        int edit_result = edit_from_array(file_path, len, to_edit, edit_len);
        if (edit_result == -2) printf("\nПроизошла ошибка чтения. Не
все строки были изменены");
        else if (edit_result == -1) printf("\nИзменение отменено по причине
недостатка прав");
        else if (edit_result == 0) printf("\nИзменение завершено");
        else if (edit_result == 1) printf("\nИзменение отменено");
        free(to_edit);
    }
    return 1;
}

// Меню "Удаление записей"
int delete_menu(char *file_path, Flight_filter filters[]) {
    system("cls");
    int len, fil_len;
    print_flights(filters, &len, &fil_len, 1);
    if (fil_len == 0) return 0;

    printf("\nВведите 0, чтобы удалить все вышепоказанные записи.");
    printf("\nВведите номера записей (1-%d), которые хотите удалить, через
запятую: ", len);
    int* to_del; // Массив индексов на удаление
    int res; // Результат ввода
    res = int_array_input(&to_del, len);

    // Обработка ошибок
    if (res == -1) printf("\nОшибка выделения памяти.");
    else if (res == 1) printf("\nУдаление записей отменено.");
    else if (res == 2 || res == 3) printf("\nНекорректный ввод.");

    else if (to_del[0] == 0) { // Если это "полный массив"
        free(to_del);
        // Просим у пользователя подтверждение операции
        printf("\nВы уверены, что хотите удалить все эти записи?\n");
        printf("\n0 - Отмена\n");
        printf("1 - Удалить\n");
        int confirm; // Буфер для ввода
        if (int_input(&confirm) || confirm != 1) printf("\nУдаление
отменено.");
        else {
            int delete_result = delete_all_filtered(file_path, filters);
            if (delete_result == -2) printf("\nПроизошла ошибка
чтения. Не все строки были удалены");
            else if (delete_result == -1) printf("\nУдаление отменено по
причине недостатка прав");
            else if (delete_result == 0) printf("\nУдаление завершено");
        }
    }
    else { // Просим у пользователя подтверждение операции и заодно считаем кол-
во элементов
        int del_len;
        printf("\nВы уверены, что хотите удалить следующие записи:\n%d",
to_del[0]);
        for (del_len = 1; to_del[del_len]; del_len++) printf(", %d",
to_del[del_len]);
        printf("\n0 - Отмена\n");
    }
}

```

```

        printf("1 - Удалить\n");
        int confirm; // Буфер для ввода
        if (int_input(&confirm) || confirm != 1) printf("\nУдаление
отменено.");
        else { // Удаляем все указанные записи
            int delete_result = delete_from_array(file_path, len, to_del,
del_len);
                if (delete_result == -2) printf("\nПроизошла ошибка
чтения. Не все строки были удалены");
                else if (delete_result == -1) printf("\nУдаление отменено по
причине недостатка прав");
                else if (delete_result == 0) printf("\nУдаление завершено");
            }
            free(to_del);
        }
    }

// Меню "Установка фильтров"
int filters_menu(Flight_filter filters[]) {
    if (print_filters(filters) == 0) return 0;
    printf("\n\n");
    printf("Введите 0, чтобы вернуться в главное меню.\n");
    printf("Введите номер столбца, чтобы настроить ему фильтр: ");
    int filters_num = 0; // Кол-во фильтров (считаем позже)
    int n, res; // Выбранный столбец и результат ввода
    res = int_input(&n);
    if (res || n == 0) return 1;

    // Считаем, сколько фильтров по указанному полю
    for (int fil = 0; fil < 2; fil++)
        filters_num += filters[fil].apply[n - 1];

    printf("\n\nВ любой момент вы можете ввести %s для отмены.\n", CANCEL);
    if (filters_num) { // Если есть хоть один фильтр
        printf("\nВведите 0, чтобы удалить фильтр по этому столбцу.\n");
        for (int fil = 0; fil < 2; fil++) {
            if (filters[fil].apply[n - 1]) {
                printf("Введите %d, чтобы изменить фильтр \"%", fil + 1);
                print_filter(filters[fil], n - 1, 1);
                printf("\n\n");
            }
            else printf("Введите %d, чтобы добавить фильтр \"%d\n", fil + 1,
fil + 1);
        }
        int choice;
        do res = int_input(&choice);
        while (res > 1 || (choice < 0 && filters_num < choice));
        if (res == 1); // Выход по ###
        else if (choice == 0) {
            for (int fil = 0; fil < 2; fil++)
                filters[fil].apply[n - 1] = 0;
            printf("\nФильтр удалён.");
        }
        else set_filter(&filters[choice - 1], n - 1, (choice - 1) == 0);
    }
    else set_filter(&filters[0], n - 1, 1); // Иначе сразу устанавливаем фильтр
    return 2;
}

```

П.А.9 Код Mark_flights.cpp

```
#define _CRT_SECURE_NO_WARNINGS
```

```

#include <stdio.h>           // Для printf, FILE, fclose
#include <conio.h>           // Для _getch
#include <stdlib.h>          // Для system, free
// Все доп. файлы
#include "functions.h"
#include "structure.h"
#include "input_funcs.h"
#include "file_funcs.h"

int main() {
    // Подготовка пользователя
    char file_path[FILE_NAME_LEN]; // Путь до файла
    if (user_prepare(file_path) == -1) {
        printf("\nДальнейшая работа невозможна\n");
        return 1;
    }

    // Создаём фильтры
    Flight_filter filters[FILTERS_NUM];
    for (int fil = 0; fil < FILTERS_NUM; fil++)
        for (int i = 0; i < FIELDS_NUM; i++)
            filters[fil].apply[i] = filters[fil].logic[i] = 0; // Обнуляем
    фильтр

    int run = 1; // Программа работает
    while (run) {
        int first_action = get_first_action(filters);
        // 1 - Вывод таблицы
        if (first_action == 1) {
            system("cls");
            int len, fil_len;
            print_flights(filters, &len, &fil_len, 1);
            printf("\nНажмите любую кнопку, чтобы продолжить");
            _getch();
        }
        // 2 - Ввод и корректировка списка
        else if (first_action == 2) {
            int edit_action = get_edit_action(filters);
            // 1 - Ввод таблицы
            if (edit_action == 1) {
                int n; // Сколько строчек нужно добавить
                int res; // Результат ввод
                FILE* table;
                do printf("\nВведите количество строчек, которые хотите
добавить (1-99): ");
                while ((res = int_input(&n)) > 1 || (n < 1 || 99 < n));
                if (res == 1) printf("\nВвод строк отменён.");
                else if ((table = fopen(file_path, "a")) == NULL)
                    printf("\nНевозможно открыть файл.");
                else {
                    int result = 0; // Результат ввода последней
                    строчки

                    int counter = 0; // Сколько записей добавили
                    for (int i = 0; !result && i < n; i++) {
                        Flight new_line;
                        result = input_flight(&new_line);
                        if (!result) {
                            write_line(table, new_line);
                            counter++;
                        }
                    }
                    fclose(table);
                }
            }
        }
    }
}

```

```

        printf("\nВвод строк закончен. Добавлено строк: %d",
counter);
    }
    printf("\nНажмите любую кнопку, чтобы продолжить");
    _getch();
}
// 2 - Изменение записей
else if (edit_action == 2) {
    edit_menu(file_path, filters);
    printf("\nНажмите любую кнопку, чтобы продолжить");
    _getch();
}
// 3 - Удаление записей
else if (edit_action == 3) {
    delete_menu(file_path, filters);
    printf("\nНажмите любую кнопку, чтобы продолжить");
    _getch();
}
// 4 - Вернуться в главное меню
else if (edit_action == 4);
}
// 3 - Установка фильтров
else if (first_action == 3) {
    filters_menu(filters);
    printf("\nНажмите любую кнопку, чтобы продолжить");
    _getch();
}
// 4 - Изменить путь до файла
else if (first_action == 4) {
    system("cls");
    if (change_path() == -1) {
        printf("\nДальнейшая работа невозможна\n");
        return 1;
    }
    int get_res;
    if ((get_res = get_file(file_path)) == -1) {
        printf("Ошибка получения прав доступа к файлу %s\n",
DIR_NAME "/" CONF_NAME);
        printf("\nДальнейшая работа невозможна\n");
        return 1;
    }
    printf("\nНажмите любую кнопку, чтобы продолжить");
    _getch();
}
// 5 - Выход из программы
else if (first_action == 5) {
    int cur_filters_num = 0;
    for (int fil = 0; fil < FILTERS_NUM; fil++)
        for (int i = 0; i < FIELDS_NUM; i++)
            cur_filters_num += filters[fil].apply[i];
    if (cur_filters_num) { // Если есть хоть один фильтр
        // Просим у пользователя подтверждение операции
        printf("\nВы уверены, что хотите выйти?\n");
        printf("Все выбранные фильтры будут потеряны\n");
        printf("\n0 - Отмена\n");
        printf("1 - Выход\n");
        int confirm; // Буфер для ввода
        if (!int_input(&confirm) && confirm == 1) run = 0;
    }
    else run = 0;
}
}
}
}

```

Приложение Б. Тесты программы

Результаты работы программы

- Первый запуск программы:

```
Mark Company Flights
Данные о полётах хранятся в файле table.csv.
Путь по умолчанию: "flights_data/table.csv". Хотите поменять путь? (вы сможете изменить его позже)
0 - Нет
1 - Да
0
Нажмите любую кнопку, чтобы продолжить:
```

- Иначе:

```
Mark Company Flights
Убедитесь, что папка flights_data используется только программой.
Нажмите любую кнопку, чтобы продолжить:
```

- Главное меню при пустой таблице:

```
Выберите действие:
1 - Вывод таблицы
2 - Ввод и корректировка списка
3 - Установка фильтров
4 - Изменить путь до файла
5 - Выход из программы
Ваш выбор: █
```

- Меню ввода и корректировки списка при пустой таблице:

```
Выберите действие по вводу и корректировке:
1 - Ввод таблицы
2 - Изменение записей
3 - Удаление записей
4 - Вернуться в главное меню
Ваш выбор:
```

- Вывод, изменение, удаление и установка фильтров при пустой таблице:

```
Таблица пустая

Нажмите любую кнопку, чтобы продолжить
```

- Ввод строки:

```

Выберите действие по вводу и корректировке:
1 - Ввод таблицы
2 - Изменение записей
3 - Удаление записей
4 - Вернуться в главное меню
Ваш выбор: 1

Введите количество строчек, которые хотите добавить (1-99): 1

Введите запись по столбцам. Чтобы закончить ввод, введите "###"
Введите номер рейса (>0): 121

Введите тип самолёта (без знака ";"): Ту-154

Введите пункт назначения (без знака ";"): Москва

Введите дни отправления (1-7) через запятую: 1,3,5

Введите время вылета (например, 13:00): 10.00

Введите время прилёта (например, 14:00): 14.00

Введите цену билета (>0): 5000.20

Ввод завершён, запись добавлена в таблицу.

Ввод строк закончен. Добавлено строк: 1
Нажмите любую кнопку, чтобы продолжить.

```

- Главное меню с непустой таблицей:

```

Авиарейсы Mark company:
Список авиарейсов:

```

Номер в таблице	Номер рейса	Тип самолёта	Пункт назначения	Дни отправления	Время вылета	Время прилёта	Цена билета
1	121	Ту-154	Москва	1,3,5	10:00	14:00	5000.20

```

Выберите действие:
1 - Вывод таблицы
2 - Ввод и корректировка списка
3 - Установка фильтров
4 - Изменить путь до файла
5 - Выход из программы
Ваш выбор:

```

- Изменение пути:

```

Текущий путь до файла flights_data/table.csv
Данные о полётах хранятся в файле table.csv.
Введите новый адрес файла, например flights_data/table.csv. Максимальная длина 127 символов
table.csv
УСПЕШНО: файл по указанному адресу существует

Нужно ли перенести информацию из прошлого файла в новый?
0 - Нет
1 - Да
1

Нужно ли удалить прошлый файл?
0 - Нет
1 - Да
0

Нажмите любую кнопку, чтобы продолжить

```


- Изменение записей:

Список авиарейсов:

Номер в таблице	Номер рейса	Тип самолёта	Пункт назначения	Дни отправления	Время вылета	Время прилёта	Цена билета
1	121	Ту-154	Москва	1,3,5	10:00	14:00	5000.20
2	1	Тульчик-номер-1	Питер	1,2	13:00	14:00	400.00
3	2	Дутик	Москва	2,3	12:00	15:00	400.00
4	3	Кролик	Тула	1,2,3,4,5,6,7	12:00	23:00	350.00
5	4	Кролик	Владивосток	3,4,5	0:00	23:59	400.00

Введите 0, чтобы изменить все вышепоказанные записи.
Введите номера записей (1-5), которые хотите изменить, через запятую: 3, 4

Для изменения выбраны следующие записи:
3, 4
Введите номера столбцов, которые вы хотите изменить, через запятую:
1 - Номер рейса
2 - Тип самолёта
3 - Пункт назначения
4 - Дни отправления
5 - Время вылета
6 - Время прилёта
7 - Цена билета
Ваш выбор: 2

Введите тип самолёта (без знака ";"): Зайчик

Строка 3 успешно изменена
Строка 4 успешно изменена

Изменение завершено
Нажмите любую кнопку, чтобы продолжить.

Список авиарейсов:

Номер в таблице	Номер рейса	Тип самолёта	Пункт назначения	Дни отправления	Время вылета	Время прилёта	Цена билета
1	121	Ту-154	Москва	1,3,5	10:00	14:00	5000.20
2	1	Тульчик-номер-1	Питер	1,2	13:00	14:00	400.00
3	2	Зайчик	Москва	2,3	12:00	15:00	400.00
4	3	Зайчик	Тула	1,2,3,4,5,6,7	12:00	23:00	350.00
5	4	Кролик	Владивосток	3,4,5	0:00	23:59	400.00

- Установка фильтра:

Список авиарейсов:

Номер в таблице	Номер рейса	Тип самолёта	Пункт назначения	Дни отправления	Время вылета	Время прилёта	Цена билета
1	121	Ту-154	Москва	1,3,5	10:00	14:00	5000.20
2	1	Тульчик-номер-1	Питер	1,2	13:00	14:00	400.00
3	2	Зайчик	Москва	2,3	12:00	15:00	400.00
4	3	Зайчик	Тула	1,2,3,4,5,6,7	12:00	23:00	350.00
5	4	Кролик	Владивосток	3,4,5	0:00	23:59	400.00

Текущие фильтры:
1. Номер рейса - не используется
2. Тип самолёта - не используется
3. Пункт назначения - не используется
4. Дни отправления - не используется
5. Время вылета - не используется
6. Время прилёта - не используется
7. Цена билета - не используется

Введите 0, чтобы вернуться в главное меню.
Введите номер столбца, чтобы настроить ему фильтр: 2

В любой момент вы можете ввести ### для отмены.

Введите тип самолёта (без знака ";"): ик

Фильтр успешно добавлен.
Нажмите любую кнопку, чтобы продолжить.

Список авиарейсов:

Номер в таблице	Номер рейса	Тип самолёта	Пункт назначения	Дни отправления	Время вылета	Время прилёта	Цена билета
2	1	Тульчик-номер-1	Питер	1,2	13:00	14:00	400.00
3	2	Зайчик	Москва	2,3	12:00	15:00	400.00
4	3	Зайчик	Тула	1,2,3,4,5,6,7	12:00	23:00	350.00
5	4	Кролик	Владивосток	3,4,5	0:00	23:59	400.00

Текущие фильтры:

- 1. Номер рейса - не используется
- 2. Тип самолёта - ик
- 3. Пункт назначения - не используется
- 4. Дни отправления - не используется
- 5. Время вылета - не используется
- 6. Время прилёта - не используется
- 7. Цена билета - не используется

Введите 0, чтобы вернуться в главное меню.
Введите номер столбца, чтобы настроить ему фильтр:

- Удаление всех записей по фильтру:

Список авиарейсов:

Номер в таблице	Номер рейса	Тип самолёта	Пункт назначения	Дни отправления	Время вылета	Время прилёта	Цена билета
2	1	Тульчик-номер-1	Питер	1,2	13:00	14:00	400.00
3	2	Зайчик	Москва	2,3	12:00	15:00	400.00
4	3	Зайчик	Тула	1,2,3,4,5,6,7	12:00	23:00	350.00
5	4	Кролик	Владивосток	3,4,5	0:00	23:59	400.00

Введите 0, чтобы удалить все вышепоказанные записи.
Введите номера записей (1-5), которые хотите удалить, через запятую: 0

Вы уверены, что хотите удалить все эти записи?

0 - Отмена
1 - Удалить
1

Удаление завершено
Нажмите любую кнопку, чтобы продолжить

Нет записей, подходящих данным фильтрам

Нажмите любую кнопку, чтобы продолжить.

- Удаление фильтра, чтобы посмотреть полную таблицу:

Нет записей, подходящих данным фильтрам

Текущие фильтры:

- 1. Номер рейса - не используется
- 2. Тип самолёта - ик
- 3. Пункт назначения - не используется
- 4. Дни отправления - не используется
- 5. Время вылета - не используется
- 6. Время прилёта - не используется
- 7. Цена билета - не используется

Введите 0, чтобы вернуться в главное меню.
Введите номер столбца, чтобы настроить ему фильтр: 2

В любой момент вы можете ввести ### для отмены.

Введите 0, чтобы удалить фильтр по этому столбцу.
Введите 1, чтобы изменить фильтр "ик"
Введите 2, чтобы добавить фильтр №2
0

Фильтр удалён.
Нажмите любую кнопку, чтобы продолжить.

Список авиарейсов:

Номер в таблице	Номер рейса	Тип самолёта	Пункт назначения	Дни отправления	Время вылета	Время прилёта	Цена билета
1	121	Tu-154	Москва	1,3,5	10:00	14:00	5000.20

Министерство науки и высшего образования Российской Федерации
ФГБОУ ВО «Алтайский государственный технический университет
имени И.И. Ползунова»

Факультет информационных технологий
Кафедра «Прикладная математика»

ОТЗЫВ

на курсовой проект по дисциплине «Программирование»

студенту группы ПИ-54 Дворникову Марку Сергеевичу

Тема курсового проекта: «Разработка программного обеспечения для
организации списка данных об авиарейсах».

За время выполнения проекта студент Дворников М.С. изучил и применил на практике базовые знания языка Си и приобрел практический навык разработки консольных приложений по работе с наборами структурированных данных. Основные характеристики выполненной работы приведены ниже.

1. Разработанная программа соответствует поставленной задаче – да / нет.
2. Программа работоспособна - да / нет / частично.
- 3 Программа реализует все поставленные задачи - да / нет.
4. Код соответствует требованиям структурного программирования - да / нет.
5. Код хорошо документирован, читабелен - да / нет.
6. Программа имеет дружелюбный для пользователя интерфейс - да / нет.
7. Пояснительная записка соответствует требованиям - да / нет / частично.
8. Курсовой проект выполнен и сдан в установленные сроки - да / нет.
9. Отмеченные достоинства:

10. Отмеченные недостатки:

Работа оценена на «_____» (____)

Руководитель проекта _____ Егорова Е.В., доцент

Дата

«02» февраля 2026 г.

Дворников Марк Сергеевич

ФИТ

ПИ-54

20 мая 2026

Доценту

кафедры ПМ

к.т.н, доценту

Егоровой Е.В.

Заявление

Прошу выставить мне промежуточную аттестацию по курсовому проекту по дисциплине
«Программирование» по результатам семестрового рейтинга.

Подпись